

Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

Anonymous authors

Paper under double-blind review

Abstract

Sequential approximate Bayesian computation (ABC) advances in generations, and a generation cannot end before its slowest simulation. On a parallel machine this barrier idles every worker that finishes early, and for identically distributed simulation times it costs the straggler factor of the workload: the expected maximum of W times over their mean. We introduce a generation-free ABC sampler. It refits its proposal after every arriving evaluation from the best particles evaluated so far, weights each particle against the mixture of proposals it could have come from, and estimates the posterior afterwards from everything that was simulated. We prove that the estimator targets the smooth-ABC posterior tilted by a ratio that measures how faithfully its denominator represents the proposals used, with exact consistency when that ratio tends to one. Empirically, it is calibrated where a pyABC baseline with the same kernel and schedule under-covers. The barrier’s cost, predicted from the asynchronous run’s timing alone, matches the same sampler run with a barrier over measured costs from $1.2\times$ to $400\times$. On a Cellular Potts tissue simulator the asynchronous sampler completes $2.1\text{--}2.4\times$ the simulations of pyABC in the same hour and ends at a $4.1\times$ tighter tolerance. With the starting bandwidth set from the prior-predictive discrepancies, the method recovers a two-parameter tissue posterior tighter than pyABC’s on a larger effective sample.

1 Introduction

Approximate Bayesian computation (ABC) is the classical route to simulation-based inference (Cranmer et al., 2020): Bayesian inference when a simulator can be run but its likelihood cannot be evaluated (Beaumont, 2019). Its sequential forms, ABC-SMC and ABC-PMC (Sisson et al., 2007; Toni et al., 2009; Beaumont et al., 2009; Del Moral et al., 2012), move a population of particles through a sequence of shrinking tolerances. Every implementation we know of does so in generations: the proposal and the tolerance are updated once the whole population has been simulated. That is a synchronization barrier. On a parallel machine with W workers a generation ends when its slowest simulation does, and every worker that finished earlier waits. If simulation times T_i are drawn from a distribution with mean μ , a generation of W takes about $\mathbb{E}[\max_{i \leq W} T_i]$. The barrier therefore costs a factor $\mathbb{E}[\max_{i \leq W} T_i]/\mu$, the *straggler factor* of the workload, which is 1 for a homogeneous simulator and grows with W and with the spread of the runtime distribution. Simulators worth running on such machines are rarely homogeneous: the cost of a cell-based tissue simulation tracks the number of cells it produces, which varies by orders of magnitude across a prior.

We remove the generation: our sampler refits its proposal mixture after every arriving evaluation, from a sliding archive of the k best particles evaluated so far, and hands the next candidate to whichever worker is free. Because the proposal has moved by the time a result arrives, every particle is weighted against the mixture of proposals actually used, the streaming limit of adaptive multiple importance sampling (AMIS) (Cornuet et al., 2012). Every quantity the sampler uses, including the posterior estimate, is recomputed from the history of evaluations, so there is no shared state to synchronize: workers never wait for one another, a

crashed run is recovered by replaying its log, and the sampler runs unchanged on the island model of the Propulate framework (Taubert et al., 2023).

The practical comparison is with pyABC (Klinger et al., 2018) run with the same kernel and bandwidth schedule; under a persistent straggler its throughput falls more than a hundredfold and ours by less than half.

Our contributions are the following; C1–C4 label the four claims that §6 tests, one per subsection.

1. A generation-free ABC sampler, smooth-kernel ABC with streaming AMIS reweighting, whose posterior estimate can be replayed from the log of evaluations (§3).
2. A prediction of the barrier’s cost from the asynchronous run’s timing alone, validated against the same sampler run with a barrier over two and a half orders of magnitude of that cost (C1, §6.1).
3. The measurement that the throughput gained converts into a tighter tolerance against pyABC on a tissue simulator, and that the advantage turns on at a few milliseconds per simulation (C2, §6.2; C3, §6.3).
4. A martingale analysis of the estimator showing that it targets the smooth-ABC posterior tilted by a ratio that measures how faithfully its denominator represents the proposals used, with exact consistency when the ratio tends to one. We derive from the execution model the adaptedness that asynchronous execution requires, and decompose the ratio and measure part of it (§4; proofs and a deadline-censoring bound in the appendix). Calibration checks in one and four dimensions and under multimodality, recovery of a two-parameter Cellular Potts posterior, and the two settings that bound the posterior, the starting bandwidth and the archive size (C4, §6.4).

2 Background and Related Work

ABC replaces the likelihood with simulation and a discrepancy: for observed data y , parameter θ and a discrepancy ρ between simulated and observed summaries, the ABC target at bandwidth ϵ is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta) L_\epsilon(\theta), \quad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho|\theta)}[K_\epsilon(\rho)], \quad (1)$$

with π the prior and K_ϵ the ABC kernel. The classical kernel is the hard indicator $\mathbf{1}[\rho < \epsilon]$, under which a simulation is either accepted or rejected and the output is a population of accepted particles. A smooth kernel such as the Gaussian $\exp(-\rho^2/2\epsilon^2)$ (Wilkinson, 2013; Fearnhead & Prangle, 2012) instead gives every simulation a graded weight and removes the discontinuity in the likelihood approximation. Throughout, ϵ is the bandwidth of this kernel; the *tolerance* a sampler has reached is a separate, empirical quantity defined in §5.3.

Sequential ABC and the generation. Drawing from the prior wastes most simulations, so the sequential family (ABC-SMC, Sisson et al., 2007; Toni et al., 2009; ABC-PMC, Beaumont et al., 2009; adaptive variants, Del Moral et al., 2012; Drovandi & Pettitt, 2011; Lenormand et al., 2013) moves a population of N particles through a sequence of shrinking bandwidths. Generation t fits a mixture proposal q_t to the weighted population of generation $t - 1$, draws candidates from it until N have been accepted at the current bandwidth, and weights each accepted particle by $\pi(\theta_i)/q_t(\theta_i)$. The next proposal needs the whole weighted population, so generation $t + 1$ cannot begin before the last candidate of generation t has been simulated. On a parallel machine that is the barrier of §1: pyABC (Klinger et al., 2018; Schälte et al., 2022), ABCpy (Dutta et al., 2021) and ELFI (Lintusaari et al., 2018) parallelize the simulations within a generation and wait at its end.

Reusing every draw: AMIS. Adaptive multiple importance sampling (AMIS) (Cornuet et al., 2012) removes the dependence of each weight on one proposal. After T stages with proposals $q_1, \dots, q_T$, every past particle is reweighted against the mixture of all of them,

$$w_i = \frac{\pi(\theta_i)}{\frac{1}{T} \sum_{t=1}^T q_t(\theta_i)} \quad \text{instead of} \quad w_i = \frac{\pi(\theta_i)}{q_{t(i)}(\theta_i)}, \quad (2)$$

the balance heuristic of [Veach & Guibas \(1995\)](#). A particle’s weight then no longer depends on which stage happened to propose it, and particles from every stage contribute to the estimate. AMIS belongs to the adaptive-importance-sampling line of population Monte Carlo ([Cappé et al., 2004](#)), surveyed by [Bugallo et al. \(2017\)](#), and is consistent under regularity conditions ([Marin et al., 2019](#)). Our method takes its streaming limit: a stage is one draw, the proposal is refitted after every arrival, and the mixture in the denominator is the sequence of proposals the run used.

Simulators with parameter-dependent cost. The simulators that motivate this work are cell-based and agent-based models of tissues: stochastic lattice or off-lattice models whose likelihood is intractable and which are calibrated against imaging data through summary statistics such as cell counts and tissue shape, typically by ABC ([Kursawe et al., 2018](#); [Lambert et al., 2018](#)). The Cellular Potts model ([Graner & Glazier, 1992](#); [Hirashima et al., 2017](#)) is the canonical lattice model of this class. Its cost scales with the number of cells the simulation produces, and that number is set by the parameters being inferred, so simulation times vary by orders of magnitude across the prior: the runtime distribution of an ABC run on such a model is wide by construction, which is where a generation barrier costs most (§5.1).

Barrier-free sampling elsewhere. Parallel and off-barrier SMC has been studied for state-space models: the island particle model ([Vergé et al., 2015](#)), asynchronous anytime particle methods ([Paige et al., 2014](#)), and parallel resampling ([Murray et al., 2016](#)). None of these targets ABC, and none combines barrier-free execution with AMIS-style reweighting against the mixture of proposals used. Look-ahead scheduling ([Alamoudi et al., 2024](#)) starts the next generation early on idle workers and so recovers part of the wait, but keeps the generation. Sequential neural simulation-based inference ([Papamakarios & Murray, 2016](#); [Greenberg et al., 2019](#); [Lueckmann et al., 2021](#)) also proceeds in rounds, each of which simulates a batch from the current proposal and retrains before the next is proposed, so every round waits for its slowest simulation as a generation does; the sampler we propose is specific to ABC. The same wait is the straggler problem of synchronous stochastic gradient descent ([Dean et al., 2012](#); [Chen et al., 2016](#)), met there with backup workers or with asynchronous, staleness-tolerant updates ([Recht et al., 2011](#)). Here the estimator is defined on whatever history has arrived, and the order of arrival matters only through the conditional law of each draw and how faithfully the posterior denominator represents it (§4).

3 Method

3.1 Overview

Figure 1 sets the method beside one generation of ABC-PMC. ABC-PMC proposes a population from one mixture, simulates it, waits for the last simulation, and only then forms the next proposal (row a). Our sampler has no population and no generation (row b). Every evaluation that completes is appended to a growing history (step 1). The k particles with the smallest discrepancies form the *archive*, and the bandwidth of the smooth ABC kernel is tightened as the archive improves (step 2). A Gaussian mixture centered on the archive members, each weighted by its fit at the current bandwidth, is the proposal (steps 3 and 4). One candidate is drawn from it and handed to the worker that has just become free (step 5). By the time that candidate’s result arrives the proposal has changed, so the candidate is weighted against the mixture of proposals it could have come from; this is the balance heuristic of AMIS (§2) applied one draw at a time (step 6). The posterior is not read off a final population: after the run, every evaluated particle is reweighted against the proposals that were in use over the whole run.

Every quantity the sampler uses is recomputed from the history it is handed, so each worker updates from whatever history it has received and the posterior estimate is the same whether it is formed during the run or afterwards. The one exception is a small buffer of past proposals that the running sampler keeps for the weight of step 6; it affects which parents are chosen next, not the estimate.

The *archive weights* $\tilde{W}_j$ shape the next proposal. The *parent weight* w^* stored with each particle is used only inside the running sampler. Only the *posterior weights* $W_{i,n}$, computed afterwards over the whole history, are reported. Sections 3.2 to 3.5 follow the figure’s steps; §3.6 assembles them.

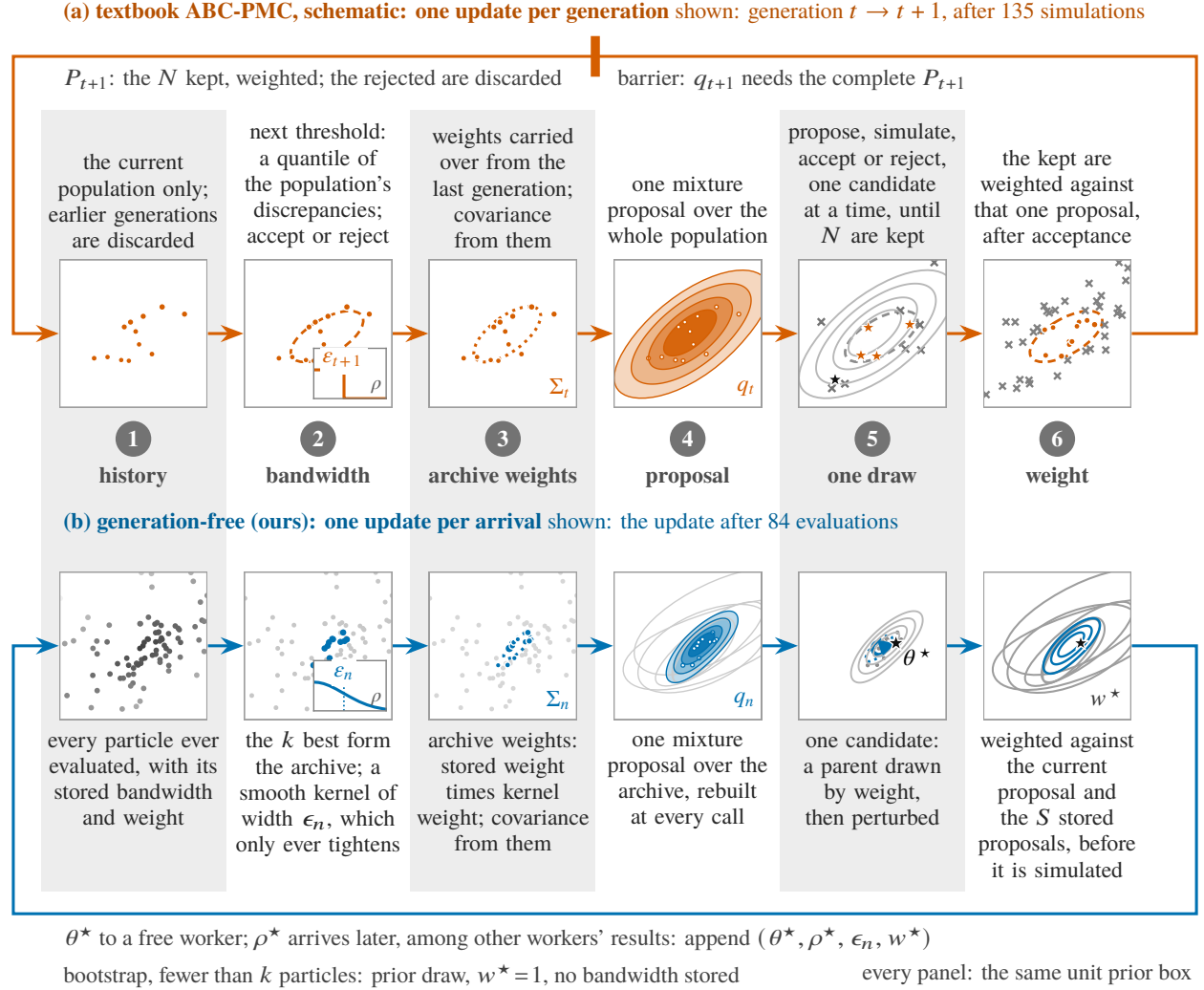


Figure 1: One generation of ABC-PMC (a) beside one update of our sampler (b), step by step on a two-dimensional toy problem. ABC-PMC proposes a whole population from one mixture, simulates all of it, keeps the accepted particles and only then forms the next proposal, so a generation ends with its slowest simulation. Our sampler appends every completed evaluation to a history, rebuilds the archive, the bandwidth and the proposal from that history at every call, draws one candidate and hands it to the free worker; the candidate is weighted against the current and the stored proposals rather than against a single one. Row (a) is a schematic of textbook hard-threshold PMC, not the smooth-kernel pyABC baseline of §5.2. The glyphs are explained in Appendix C.

3.2 History and archive (steps 1 and 2)

Write the history at call n as

$$\mathcal{H}_n = \{(\theta_i, \rho_i, \epsilon_i, w_i)\}_{i=1}^n, \quad (3)$$

where θ_i is the parameter, ρ_i the discrepancy, ϵ_i the bandwidth in force when θ_i was proposed, and w_i its parent weight (§3.5). The archive A_n is the set of the k history members with the smallest discrepancies. Until the history holds k members the sampler emits uniform prior draws, with $w_i = 1$ and no bandwidth, so that the first archive can form. The history is append-only, and the archive, the bandwidth and the proposal below are recomputed from it at every call. The Propulate framework (Taubert et al., 2023), which hands the whole history to each call and takes one candidate back, provides this contract (Appendix C).

3.3 Bandwidth (step 2)

The smooth kernel K_ϵ of (1) replaces the accept-or-reject threshold of classical ABC, and the bandwidth ϵ_n sets how sharply the archive weights and the posterior concentrate. The bandwidth tightens as the archive improves and never loosens, so that every worker, whatever prefix of the history it holds, is on the same tightening sequence. The bandwidth in force at call n is the smaller of the running minimum of the bandwidths stored in the history and the value a scheduler proposes from that history. The scheduler starts from a user-set ϵ_0 . At each search it proposes the largest bandwidth at which the kernel-weighted effective sample size of the archive retains 95% of its current value, by at most a factor of two, and it holds the bandwidth until $2k$ evaluated particles lie within it, so that ϵ_n settles near that order statistic of the discrepancies (settings in Appendix C). On an expensive simulator the run can end before the schedule has moved far from ϵ_0 , so ϵ_0 has to be set from the scale of the prior-predictive discrepancies (§5.3, §6.4).

3.4 Archive proposal (steps 3 to 5)

The proposal at call n is a Gaussian mixture over the archive,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_\Sigma(\theta - \theta_j), \quad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \quad (4)$$

with $\sum_j \tilde{W}_j^{(n)} = 1$: the archive weight of a member is its parent weight times its kernel value at the current bandwidth. The perturbation kernel K_Σ is Gaussian with covariance $\Sigma_n = s \widehat{\text{Cov}}_{\tilde{W}}(A_n)$, the weighted covariance of the archive scaled by a constant s , and each component is truncated to the prior box. A candidate is drawn by choosing a parent in proportion to $\tilde{W}^{(n)}$ and perturbing it. Because the kernel is smooth, an archive member's contribution shrinks continuously as the bandwidth tightens instead of dropping out when a threshold crosses it, as it does in hard-threshold ABC-PMC. The numerical details are in Appendix C.

3.5 Weights and the posterior estimate (step 6)

A candidate drawn from q_n should be importance-weighted against the proposal it came from, but the proposal moves with every arrival, so the history was drawn from a long sequence of proposals. AMIS resolves this by weighting against the mixture of the proposals used (§2). A *proposal snapshot* q_s is the proposal (4) rebuilt from the history truncated at call s , at the bandwidth stored there; any snapshot can be rebuilt from the history alone.

The parent weight. The running sampler keeps the last S snapshots it has taken, one every k calls. A candidate θ^* drawn from q_n receives

$$w^* = \frac{\pi(\theta^*)}{\bar{q}_n^{\text{on}}(\theta^*)}, \quad \bar{q}_n^{\text{on}}(\theta) = \frac{1}{1 + |\mathcal{S}|} \left[q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta) \right], \quad (5)$$

with $\mathcal{S}$ the kept snapshots; $S = 0$ recovers single-proposal weighting. This weight is stored with the particle as its w_i and enters the archive weights of (4) and the scheduler's effective-sample-size search (§3.3). The kept snapshots are the only state the sampler carries between calls, and a stale or empty buffer changes which parents are chosen, not the estimate.

The posterior weights. After the run, the estimate rebuilds its own snapshots from $\mathcal{H}_n$, at most $S+1$ of them (two when $S = 0$), at history indices $\tau_1 < \dots < \tau_m$ spread in proportion to the number of draws each stands in for, and reweights every evaluated particle against their mixture. For a function f of the parameter the estimate is

$$\hat{\pi}_n(f) = \frac{\sum_{i=1}^n W_{i,n} f(\theta_i)}{\sum_{i=1}^n W_{i,n}}, \quad W_{i,n} = \frac{\pi(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}, \quad (6)$$

with the denominator

$$\bar{q}_n = \lambda_n \pi + (1 - \lambda_n) \sum_{s=1}^m \alpha_{s,n} q_{\tau_s}, \quad \lambda_n = \max(\nu_n, \delta), \quad \alpha_{s,n} \geq 0, \quad \sum_s \alpha_{s,n} = 1. \quad (7)$$

This is the mixture of the m reconstructed snapshots, weighted in proportion to the draws each stands in for, and of the prior, whose weight λ_n is the share ν_n of prior draws in the history floored at a fixed $\delta > 0$; the prior component keeps the denominator bounded away from zero and so bounds every weight (Owen & Zhou, 2000). The snapshot mixture is a fixed-size stand-in for the mixture the sample was actually drawn from, and §4 asks how faithful a stand-in it is. Computing $\hat{\pi}_n$ is one $\mathcal{O}(nk)$ pass over the history, run after the timed budget and excluded from every throughput measurement (Appendix C).

3.6 Update loop

Algorithm 1 is the update a worker runs whenever it needs a candidate; its lines follow the steps of Fig. 1. On W workers each one runs the loop on its own copy of the history, which keeps growing as results arrive from every worker. No line waits for any other worker, and a worker that returns from a slow simulation finds a larger history than the one it left.

Algorithm 1 One update of the generation-free sampler, run by a worker on its copy of the history

Require: History $\mathcal{H}_n$, archive size k , starting bandwidth ϵ_0 , kept snapshots $\mathcal{S}$

- (1) **if** $|\mathcal{H}_n| < k$ **then return** a uniform prior draw with $w^* = 1$ ▷ bootstrap
 - (2) $A_n \leftarrow$ the k members of $\mathcal{H}_n$ with the smallest ρ_i ; $\epsilon_n \leftarrow$ the smaller of the stored running minimum and the scheduler’s proposal
 - (3) $\tilde{W}_j^{(n)} \propto w_j K_{\epsilon_n}(\rho_j)$ over A_n ; $\Sigma_n \leftarrow s \widehat{\text{Cov}}_{\tilde{W}}(A_n)$
 - (4) q_n as in (4); every k calls, add q_n to $\mathcal{S}$
 - (5) draw a parent $j \sim \tilde{W}^{(n)}$ and $\theta^* \sim K_{\Sigma_n}(\cdot - \theta_j)$, retried into the prior box
 - (6) w^* as in (5)
- return** θ^* for simulation; $(\theta^*, \rho^*, \epsilon_n, w^*)$ joins every worker’s history when ρ^* arrives
-

4 Theoretical Analysis

The estimate $\hat{\pi}_n$ of (6) divides each particle’s weight by $\bar{q}_n$, a fixed-size mixture of reconstructed proposals, whereas the particles were drawn from the sequence of all proposals the run used. Consistency therefore depends on how well $\bar{q}_n$ represents the density from which the pooled sample was drawn. Write $\tilde{q}_i$ for the conditional density of the i -th candidate given the past (the filtration of Assumption 4); it absorbs everything the implementation does, and it is the only way the sampler enters the theorem. The mixture the pooled sample was drawn from is

$$\bar{q}_n^* := \frac{1}{n} \sum_{i \leq n} \tilde{q}_i, \quad (8)$$

and the *fidelity ratio* of the run is $r_n := \bar{q}_n^* / \bar{q}_n$. We prove that the estimate tracks the smooth-ABC posterior at the realized bandwidth tilted by r_n , with an error that vanishes whatever the sampler does. When r_n has a limit r , the estimate converges to the posterior tilted by r , so that $r \equiv 1$ gives the posterior itself. The proof is self-contained rather than a corollary of AMIS theory, because the published consistency results for recycling schemes (Marin et al., 2019) restrict the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do. Appendix A gives the decomposition and partial measurement of r ; the proofs are in Appendix B. Throughout, $\epsilon_\infty := \lim_n \epsilon_n$, which exists because the schedule never loosens, and which may be zero; the asymptotic statements hold on the event $\{\epsilon_\infty > 0\}$, and the closing paragraph of this section says which configurations put a run there. $L_\epsilon(\theta) = \mathbb{E}[K_\epsilon(\rho) \mid \theta]$ is the smooth ABC likelihood of (1).

Assumptions.

Assumption 1 (kernel). $K_\epsilon : [0, \infty) \rightarrow [0, 1]$ is measurable with $K_\epsilon(0) = 1$ (unit *peak*, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and for every $a \in (0, \epsilon_0]$ there is $L_K(a) < \infty$ with $|K_\epsilon(\rho) - K_{\epsilon'}(\rho)| \leq L_K(a)|\epsilon - \epsilon'|$ for all $\rho \geq 0$ and $\epsilon, \epsilon' \in [a, \epsilon_0]$. The limiting target is non-degenerate: $Z_{\epsilon_\infty} = \int_{\Theta} \pi L_{\epsilon_\infty} > 0$.

Assumption 2 (denominator class). $\Theta \subset \mathbb{R}^d$ is a non-degenerate axis-aligned box and the prior density satisfies $0 < \pi_- \leq \pi \leq \pi_+ < \infty$ on it. Each q_{τ_s} in (7) is a positive function on Θ of truncated-Gaussian-mixture form: at most k components, means in Θ , weights in the simplex, covariance Σ with $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ for fixed $0 < \sigma_- \leq \sigma_+ < \infty$, and per-component normalizing constants evaluated as the implementation evaluates them.

Assumption 3 (prior floor in the denominator). $\delta \in (0, 1/2]$ is fixed and $\lambda_n \geq \delta$ in (7).

Assumption 4 (execution model). Candidates are indexed by proposal time. (B1) Given its parameter θ , a simulator call returns a discrepancy and a wall-clock duration, $(\rho, D) \sim P(\cdot \mid \theta)$, and the pairs of distinct calls are conditionally independent given their parameters. (B2) The sampler’s own randomness (parent choice, perturbation, rejection retries, redraws, bootstrap draws) is drawn fresh at each call from a stream independent of the simulator’s. (B3) Earlier proposal times, durations and worker identities, together with an exogenous source of randomness (message delivery), determine which worker proposes next, when it proposes, and which earlier results it has received by then. The exogenous source is independent of the streams in (B1) and (B2). Lemma 5 derives from this model the filtration $(\mathcal{F}_i)$ used in the proofs. Under that filtration, $\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i$, with $\tilde{q}_i$ the proposal density the worker emits, and $\rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i)$. Under sequential execution (B3) is empty.

Assumption 5 (fidelity). The denominator tracks the draw mixture up to a limiting ratio: for some r with $0 < r_- \leq r \leq r_+ < \infty$,

$$\|\bar{q}_n^*/\bar{q}_n - r\|_\infty \longrightarrow 0 \quad \text{a.s.} \quad (9)$$

Assumption 1 holds for the Gaussian kernel with $L_K(a) = 2/(ea)$ and Epanechnikov with $2/a$; it excludes the hard indicator, which is not Lipschitz in ϵ . This is one reason the method uses smooth kernels. Assumptions 2 and 3 constrain only the m -snapshot denominator $\bar{q}_n$, which the implementation builds to that exact recipe. Assumption 4 is a property of the execution, not of the proposal path; Table 5 records it for the implementation. Assumption 5 is the only place the sampler’s proposal path enters; Theorem 1 does not need it.

Theorem 1 (tracking at the realized bandwidth). *Let Assumptions 1–4 hold and write $r_n := \bar{q}_n^*/\bar{q}_n$. Then for every bounded measurable $h : \Theta \rightarrow \mathbb{R}$, almost surely on $\{\epsilon_\infty > 0\}$,*

$$\frac{1}{n} \sum_{i \leq n} W_{i,n} h(\theta_i) - \int_\Theta \pi h L_{\epsilon_n} r_n \longrightarrow 0. \quad (10)$$

Consequently, if in addition $\liminf_n \frac{1}{n} \sum_{i \leq n} W_{i,n} > 0$, then for every bounded measurable f ,

$$\hat{\pi}_n(f) - \pi_{\epsilon_n}^{r_n}(f) \longrightarrow 0, \quad \pi_\epsilon^r(f) := \frac{\int_\Theta f \pi L_\epsilon \varrho}{\int_\Theta \pi L_\epsilon \varrho}. \quad (11)$$

The estimate thus behaves as if its denominator were the density the sample was drawn from, up to the tilt r_n , whatever the proposal path; the theorem’s two conditions, a bandwidth away from zero and an average weight that does not collapse, are observable from a run. Passing to a limit needs the ratio to settle:

Corollary 2 (tilted limit). *Let Assumptions 1–5 hold. Then for every bounded measurable f , almost surely on $\{\epsilon_\infty > 0\}$,*

$$\hat{\pi}_n(f) \longrightarrow \pi_{\epsilon_\infty}^r(f) = \frac{\int_\Theta f \pi L_{\epsilon_\infty} r}{\int_\Theta \pi L_{\epsilon_\infty} r}. \quad (12)$$

Corollary 3 (exact fidelity). *If moreover $r \equiv 1$, that is, if the denominator is asymptotically the density the sample was drawn from, then $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}(f)$, the smooth-ABC posterior at the limiting bandwidth.*

What r is, and how much of it is measured. Everything specific to the implementation enters through r , in four factors: the prior floor δ in the denominator, the m reconstructed snapshots standing in for the n proposals actually used, the approximate normalization of the truncated mixture components, and the sampler’s departures from drawing exactly from q_n (Appendix A.1). The first two can be estimated from a stored run by recomputing the denominator at a much larger m and at the observed prior share: on the

five Cellular Potts production runs the estimate lies 0.2–0.5% in total variation from that reference, and on a one-dimensional Gaussian history 0.26%. The other two are bounded, not measured, so the accounting of r is partial and the limit statements are conditional on its finite positive limit (Table 5). The experiments use $S = 20$, so $m \leq 21$, and the floor is $\delta = 0.5/(m+1)$ (Appendix C).

Which runs the limits describe. Every asymptotic statement above holds on $\{\epsilon_\infty > 0\}$. The event concerns only the bandwidth ϵ_n at which the estimate is reported in (6); the bandwidths the sampler uses to build its proposals enter the proofs only through $\tilde{q}_i$, on which Theorem 1 places no condition. A bandwidth floor (`min_tol`, Appendix C) puts a run on the event by construction, and we recommend $10^{-4}\epsilon_0$. The floor acts only by clipping a bandwidth that would fall below it, so a run whose schedule stays above the floor follows exactly the path it would have followed with the floor set. That holds for every g-and-k and Cellular Potts run whose posterior we report, including a replayed sample of the g-and-k calibration trials: their final bandwidths lie more than ten times above $10^{-4}\epsilon_0$ (Table 5), so the limits describe these runs as they were executed. The Gaussian-mean and Lotka–Volterra benchmarks, near-instantaneous simulators that run for millions of evaluations, tighten to below $10^{-5}\epsilon_0$ and are not covered at that floor. Being covered does not make a budget asymptotic. No reported run reached its floor, so each is still a fixed- k estimate: the scheduler holds the bandwidth until $2k$ evaluated particles lie inside it (§3.3), only $O(k)$ particles carry appreciable kernel mass, and the effective-sample-size ceiling of §6.4 is the visible consequence. Once the floor is reached, the average weight converges to a positive constant and the effective sample size grows linearly in n ; the limits describe that continuation, and the measurements above and the calibration checks of §6.4 quantify the error at the budgets used.

5 Experimental Design

5.1 Benchmarks

We evaluate calibration and recovery on targets with reference posteriors, test the barrier prediction under runtime laws known by construction, and measure performance on a real simulator of the class described in §2. Table 1 lists the five configurations. The one-dimensional Gaussian mean has an analytic posterior and the four-dimensional g-and-k distribution (Rayner & MacGillivray, 2002) a reference posterior; both are near-instant simulators, so they also expose the cost of coordinating one arrival at a time. The two studies with injected runtime heterogeneity run on the Gaussian mean: a persistently slow worker, and a lognormal multiplier on every evaluation’s runtime. Lotka–Volterra is a stochastic simulator whose cost varies with the trajectory and whose parameters send 98% of simulations to extinction (Appendix D); it serves the scaling and boundary measurements and is not scored for posterior quality.

The Cellular Potts benchmark is the real simulator, and its runtime spread is a property of the inference problem rather than of the machine. How fast cells divide and how large they grow determine how many cells the lattice holds, and the cell count fixes both the summaries and the runtime (correlation 0.99 between runtime and cell count). We run the `cellsInSilico` implementation (Berghoff et al., 2020) of the three-dimensional Cellular Potts model (Graner & Glazier, 1992) end to end at two lattice sizes, and infer the two parameters the summary statistics resolve, the cell division rate and the target cell volume, from the final cell count and the cluster radius; one evaluation averages four simulations, because a single realization is too noisy to order nearby parameters. The simulations take seconds to minutes rather than the hours of a production study, and the observations are synthetic with a known truth, so the benchmark scores both the systems claims and posterior recovery. The 50^3 twin and scaling runs used an earlier parameterization of the same simulator (division rate and motility, one simulation per evaluation); only its runtime distribution enters those results. Appendix E gives the screening experiment that fixed the parameters and summaries, the prior ranges, and both configurations.

5.2 Baselines and instruments

We use three comparators, each for a different question. *pyABC* is pyABC (Klinger et al., 2018; Schälte et al., 2022) with its acceptor replaced by a probabilistic-rejection acceptor using the same smooth kernel

Table 1: Benchmark suite. Cost is the mean wall-clock time of one simulation in the asynchronous runs (an evaluation on Cellular Potts is four replicate simulations); runtime spread is the in-run coefficient of variation of that time, or how it is injected. Budgets are per replicate; five replicates throughout except three at 80^3 .

Benchmark	d	cost/sim	runtime spread	budget, workers	role
Gaussian mean	1	0.1 ms	injected: straggler 0–20×; lognormal $\sigma \leq 2$	300 s, 48 (16 in the straggler study)	calibration; barrier cost under a known runtime law; the cheap end of the cost boundary
g-and-k	4	2 ms	none	600 s, 48	4-D calibration and recovery against a reference posterior
Lotka–Volterra	4	4 ms	trajectory-dependent; 98% extinct	600 s, 48; scaling to 288	strong scaling; the cost boundary
Cellular Potts 50^3	2	13 s	CV 0.05	3600 s, 48; scaling to 384	posterior recovery against a known truth; barrier cost on a real simulator
Cellular Potts 80^3	2	170 s	CV 0.13–0.23	3600 s, 48	barrier cost at realistic cost and heterogeneity

and the same bandwidth schedule as the asynchronous sampler (Appendix C); it is the method a practitioner would run, and the comparator for C2 and C3. Its population of 100 on 47 simulating workers (one process only dispatches) forces at least three rounds per generation, the last one 13% full, so it idles by construction even on a homogeneous simulator. The scaling study gives it a population equal to the worker count; the production comparisons do not. The *twin* is our own propagator with a collective barrier inserted before each proposal and nothing else changed. Histories are not truncated to generation boundaries, so each worker proposes from the history it has received at the barrier. The twin thus isolates the timing of synchronization from every other difference, and it is the comparator for C1. A collective barrier needs identical call counts on every worker, so the twin runs to a fixed simulation count, the asynchronous sampler runs to a fixed wall clock, and the two are compared as rates. For the posterior comparison the asynchronous sampler is also re-run at exactly the twin’s simulation count and seeds. *Rejection ABC* spends the same simulation budget, keeps the k best draws, and is given the whole machine (all 48 workers) rather than one worker per replicate. It is the reference for the per-simulation gain of an adaptive proposal. All three run over MPI on JUWELS (Appendix D) and are samplers of the same estimator class; a neural simulation-based-inference baseline would change the estimator rather than the barrier, and its rounds would wait for their slowest simulation as generations do (§2).

5.3 Metrics

Tolerance at a matched budget. To compare samplers on one footing we use the k -th order statistic of a method’s own discrepancies after n simulations, $\epsilon_{(k)}(n)$: the tolerance at which exactly k of its draws would be accepted. It needs no reference posterior, it does not depend on the bandwidth a schedule happened to report at, and it is what rejection ABC reports at by construction. The *per-simulation efficiency* of the asynchronous sampler over pyABC is the ratio of their $\epsilon_{(k)}$ at the same n , above one when the asynchronous tolerance is the tighter; the *equal-wall-clock* ratio is the same quantity at the end of both runs, and the two differ by the throughput ratio. The slope of $\epsilon_{(k)}(n)$ in $\log n$ shows whether more simulations keep converting into a tighter tolerance. We use $k = 100$ throughout, over completed simulations only, ordered by completion time (Appendix D).

Throughput, utilization, and the predicted barrier cost. Throughput is simulations completed per second of wall clock; utilization is the fraction of worker time spent inside the simulator. The prediction that C1 tests is built from the asynchronous run only. Let D_i be the per-evaluation durations on that run (recorded simulation time plus the measured per-evaluation coordination overhead). A barrierized generation of W evaluations takes $\mathbb{E}[\max_{i \leq W} D_i]$, so the synchronous throughput is predicted as $T_{\text{sync}} = W/\mathbb{E}[\max_{i \leq W} D_i]$. The expectation is taken by resampling W durations from the asynchronous record, or from the injected law where the heterogeneity is injected; a persistently slow worker contributes its own mean. The predicted cost is $T_{\text{async}}/T_{\text{sync}}$ with T_{async} the measured asynchronous throughput, and it is compared with the ratio

measured against the twin. For identically distributed, uncensored durations $T_{\text{async}} = W/\mathbb{E}[D]$ and the ratio is the straggler factor of §1. With a persistently slow worker, or when a wall-clock budget censors the long durations, the two differ, and we use the general form. The prediction has a domain: the simulation time must dominate the latency of the collective itself, which the model does not contain, and the timing sample must be long enough to cover the tail of the runtime distribution. Configurations outside this domain are marked as such in §6.1.

Posterior quality. Calibration is checked by simulation-based calibration (Talts et al., 2018): over 1000 trials, each with a parameter drawn from the prior and data simulated from it, the empirical coverage of the posterior’s central intervals at the 0.50/0.80/0.90/0.95 levels. The effective sample size (ESS) of a weighted posterior is $(\sum_i W_i)^2 / \sum_i W_i^2$. Where a reference posterior exists we report the Wasserstein distance W_1 between each method’s own posterior and that reference (in more than one dimension the sliced W_1 over random projections (Bonneel et al., 2015), on the parameters’ natural scale). The reference is the analytic posterior for the Gaussian mean; for g-and-k it is the posterior given the same octile summaries the discrepancy is built from, obtained from their asymptotic law (Appendix D). The asynchronous estimator is replayed over the prefix of the history completed by each checkpoint. pyABC is scored on its last completed generation, with pyABC’s population weights where the history was kept and uniform weights otherwise, and rejection ABC on its k best draws. On Cellular Potts, whose truth is known but whose posterior is not, we report each parameter’s *contraction* $1 - \text{sd}/\text{sd}_{\text{prior}}$ in prior-normalized units, whether the central 90% interval covers the truth, and the ESS of the posterior weights.

Starting bandwidth. On an expensive simulator the bandwidth schedule may not move far from ϵ_0 within the budget (§3.3), so ϵ_0 is set before the run from the discrepancies of a few thousand prior-predictive draws, at about a fifth of their median; §6.4 measures what a starting value far above that costs.

6 Results

Unless noted, results are medians over five replicates with the replicate range shown; *asynchronous* is the proposed sampler, *pyABC* the baseline with the same kernel and schedule, and *twin* the barrierized copy of our own propagator (§5.2).

6.1 The barrier costs the straggler factor, and the cost is predictable (C1)

The prediction holds. Figure 2 covers every configuration on which we ran the barrierized twin: a persistently slow worker at five slowdowns, lognormal runtime heterogeneity at five spreads, and the Cellular Potts simulator at four worker counts and two lattice sizes. For each, the barrier’s cost as predicted from the asynchronous run’s timing alone (§5.3) is plotted against the cost measured with the twin. Over the twelve configurations inside the predictor’s domain the predicted-to-measured ratio runs from 0.84 to 1.19 with median 0.996, across measured costs from $1.2\times$ to $400\times$ (Tables 9, 10 and 11 in Appendix F). The two largest misses, 16% and 19% at the widest runtime spreads, occur where the twin completes only ten generations per replicate and the 60s budget censors the asynchronous run’s longest draws. Three configurations lie outside the domain and are drawn open: the cost-free simulator at $0\times$ and $1\times$, where the collective’s own latency, about 0.3s per barrier, dominates a 4ms workload; and 384 Cellular Potts workers, where the asynchronous timing sample carries a contention tail that the twin’s generations did not show.

What the twin does not isolate. On Cellular Potts at 50^3 the twin’s throughput was $1.9\text{--}2.8\times$ below the asynchronous sampler’s, but only part of that is the barrier. Throughput is utilization over mean simulation time, and the two factors separate: the utilization ratio is $1.2\times$ at every worker count and matches the prediction, while the rest is the twin’s simulations running longer for reasons unrelated to the parameters, which we do not attribute (Table 11). The barrier’s cost on this workload is therefore $1.2\times$, as its runtime spread predicts, and it rises with the lattice: at 80^3 , where an evaluation costs 170s and the runtime coefficient of variation is 0.23, the predicted cost is $1.9\times$ and the gap measured against pyABC $2.1\times$. The same comparison against pyABC at 50^3 gives $2.3\text{--}2.4\times$, of which only $1.2\times$ is the barrier; the rest is pyABC’s fixed per-generation overhead, a large fraction of a 13s evaluation and negligible against a 170s one. The

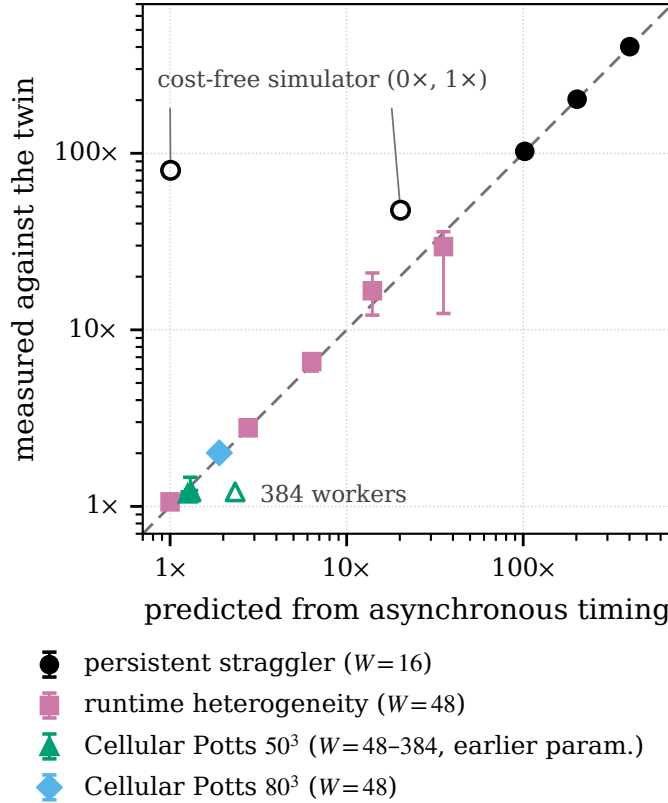


Figure 2: The barrier’s cost predicted from the asynchronous run’s timing alone (§5.3) against the cost measured with the barrierized twin, on every configuration the twin was run on: a persistent straggler at five slowdowns, lognormal runtime heterogeneity at five spreads, and Cellular Potts at 50^3 (48–384 workers, earlier parameterization, utilization ratio) and 80^3 (48 workers, against pyABC). Medians with replicate ranges; the dashed line is equality; open markers are the three configurations outside the predictor’s domain.

asynchronous sampler itself scales almost linearly on this workload, at 97% parallel efficiency from one to eight nodes (Fig. 3, right).

Against pyABC. The same experiments score pyABC (Fig. 8, Appendix F). Under the persistent straggler its throughput falls more than a hundredfold while the asynchronous sampler’s falls by less than half, because every generation waits for the slow worker. Under runtime heterogeneity its workers idle 51–79% of their time against 1–27% for ours, and its throughput is 2.1–4.8 $\times$ below ours. That is less than the twin loses, because pyABC closes a generation on the first 100 finishers and discards the latecomers it has already paid for, so the comparison a practitioner faces understates the barrier’s full cost. On Cellular Potts pyABC runs at 43–49% utilization against 98–99.8%, and the asynchronous sampler completes 2.1–2.4 $\times$ its simulations in the same budget.

6.2 Throughput converts into a tighter tolerance (C2)

Throughput matters only through the tolerance a run reaches. Table 2 reads it off every benchmark as $\epsilon_{(100)}(n)$ (curves in Fig. 9, Appendix F). At pyABC’s own simulation count the asynchronous sampler is the more efficient per simulation on three of the four benchmarks; the one on which it is worse is the one-dimensional analytic target. The product of the per-simulation ratio and the throughput ratio, the ratio at equal wall clock, exceeds one on Lotka–Volterra and reaches 4.1 $\times$ on Cellular Potts. On the two cheap benchmarks the asynchronous and pyABC curves have the same slope, so the per-simulation ratio is

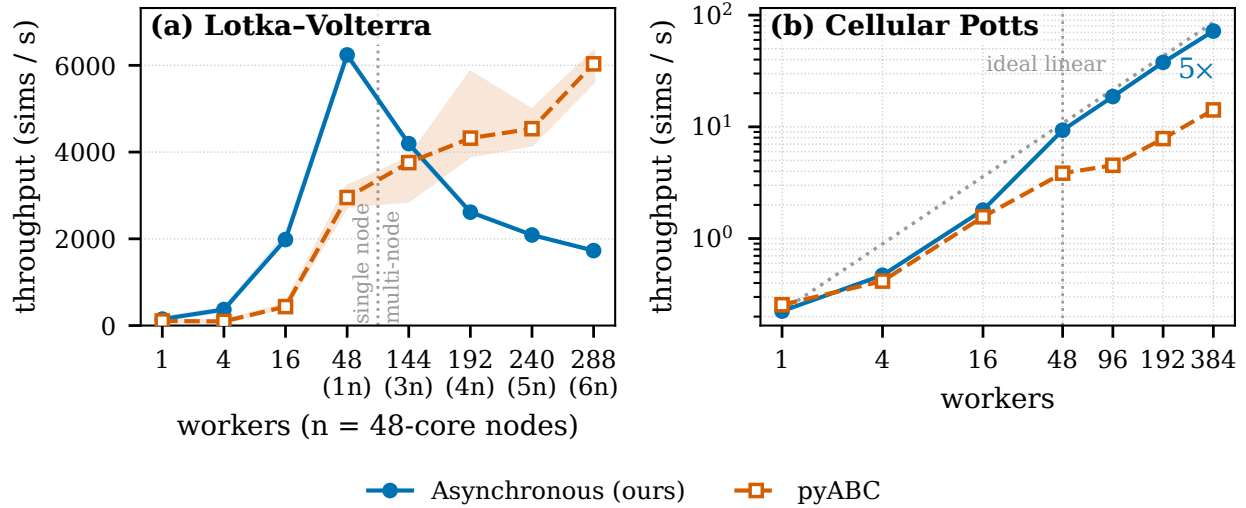


Figure 3: Strong scaling at a fixed wall-clock budget against pyABC with population $\max(100, W)$. **Left:** Lotka–Volterra (4 ms; 180 s; medians and inter-quartile ranges over five replicates). **Right:** Cellular Potts 50^3 (earlier parameterization, 5.1 s; 1800 s; log–log), with linear scaling dotted.

Table 2: Tolerance at a matched budget on every benchmark ($k = 100$ order statistic of each method’s own discrepancies; five replicates; medians, with the replicate range of each ratio in parentheses; completed simulations only). Per-simulation: ϵ of pyABC over the asynchronous sampler at pyABC’s own simulation count; equal wall clock: the same ratio at the end of both runs.

Benchmark	cost/sim	sims (async / sync)	throughput	per-simulation	equal wall clock
Gaussian mean	0.1 ms	1.31M / 4.61M	0.28×	0.78×	0.24×
g-and-k	2 ms	1.75M / 3.81M	0.46×	1.06×	0.88×
Lotka–Volterra	4 ms	1.51M / 1.56M	0.96×	1.30×	1.30×
Cellular Potts	13 s	12,900 / 5,400	2.39×	1.6×	4.09×

a constant factor; on Cellular Potts both adaptive samplers steepen well beyond the n^{-1} of rejection ABC in this noise-dominated regime, so their advantage over it widens with every doubling of the budget.

Cellular Potts, fixed production run. Table 4 gives the decomposition on the production run. In the same hour the asynchronous sampler completes $2.4\times$ the simulations of pyABC, of which §6.1 attributes $1.2\times$ to the barrier at this lattice size and most of the rest to pyABC’s per-generation overhead. It is $1.6\times$ more efficient per simulation, read at pyABC’s count of 5,400 where pyABC’s curve is the steeper (Fig. 9d), and ends $4.1\times$ tighter at equal wall clock. A rejection sampler given the same budget and the whole machine ends $39\times$ looser. At 80^3 the asynchronous sampler completes $2.1\times$ the simulations and is $2.1\times$ more efficient per simulation.

6.3 Where the advantage turns on (C3)

Each arrival incurs a roughly fixed coordination cost: rebuilding the $\mathcal{O}(k)$ proposal, weighting the arrival against the kept snapshots, and exchanging it with every other worker. This cost becomes less important as the simulation cost increases. Figure 4 plots the throughput and per-simulation ratios of Table 2 against the cost of one simulation. Both ratios increase with simulation cost. Only the throughput ratio has a common mechanism across benchmarks; the per-simulation ratio also depends on dimension, summaries and noise in the four inference problems. We therefore base the boundary on throughput: parity is reached at about 4 ms per simulation, and the net advantage at equal wall clock turns positive between 2 and 4 ms. That boundary belongs to the placement and budget it was measured under: the cheap benchmarks ran 48

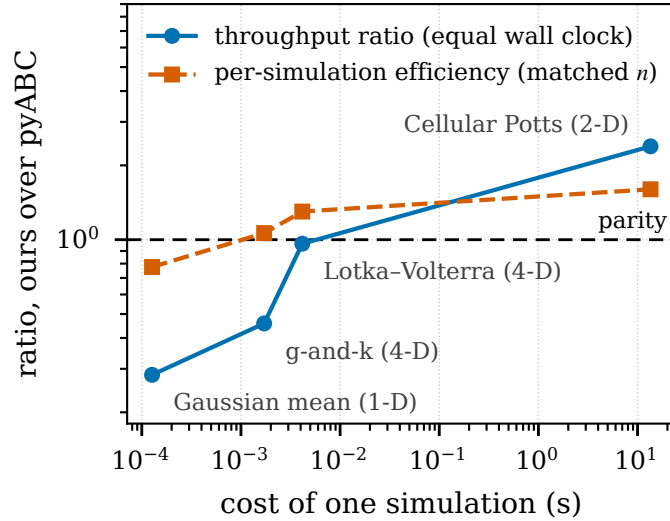


Figure 4: Throughput ratio and per-simulation efficiency of the asynchronous sampler over pyABC against the cost of one simulation, one point per benchmark (Table 2).

workers spread over four nodes (Appendix D), so every arrival crosses the network, and on one packed node the same Lotka–Volterra configuration gives a throughput ratio of 2.1 instead of 0.96 (Table 12). Below the boundary a barrier costs nothing worth removing: on the 0.1 ms Gaussian mean pyABC completes $3.5\times$ more simulations and ends with a $4\times$ tighter tolerance at equal wall clock.

The same per-arrival cost sets the scaling boundary on a cheap simulator. On Lotka–Volterra the exchange of every arrival with every worker grows with the worker count, so a worker is inside the simulator 25% of its time at 48 workers and 2% at six nodes; throughput peaks on one node, and pyABC with a population equal to the worker count overtakes beyond three nodes (Fig. 3, left). At 13 s per evaluation the same coordination is negligible, and the method tracks linear scaling to eight nodes (Fig. 3, right).

6.4 The posterior: calibration, recovery, and what limits it (C4)

Calibration. Table 3 gives simulation-based calibration of the full-history estimator (1000 trials, matched simulation budget). On the one-dimensional Gaussian mean it tracks the nominal coverage at every level, while pyABC under-covers at every level (its nominal 0.90 interval covers 0.75); the Monte Carlo error of ± 0.01 resolves this ordering. On a bimodal target ($y_i \sim \mathcal{N}(\theta^2, \sigma^2)$, the case a top- k archive is most at risk of collapsing onto one mode) the archive retains both modes in 86% of trials and coverage is nominal. On the four-dimensional g-and-k model the estimator is *conservative* on A and g and near nominal on B and k , with B ’s rank histogram the only uniform one at the 1% level. pyABC is near nominal on A and g and under-covers B and k , with no uniform rank histogram. The top- k archive alone would under-cover (0.85/0.89 at the 0.90/0.95 levels). The full history over-covers. Coverage rises monotonically through nominal as the support grows from one to the other. This pattern points to a reporting-support effect rather than a weighting failure. Doubling the simulation budget leaves the coverage unchanged. A sweep over the archive size and the number of kept snapshots on a d -dimensional Gaussian target (Appendix F) finds a miscalibrated region at $d = 8$, $k = 50$; of the archive sizes swept, $k = 100$ is the best calibrated.

Recovery against a reference posterior. Where a reference exists, Fig. 5 scores each method’s posterior against it over wall-clock prefixes. On the Gaussian mean, the asynchronous estimator is closer to the analytic posterior than pyABC’s final population at every checkpoint. A rejection sampler given the same simulation budget is as close. In one dimension with that many draws, rejection is enough. On the four-dimensional g-and-k the order reverses: pyABC ends at $W_1 = 0.029$ and the asynchronous estimator at 0.069, with rejection far behind at 0.47. The effective sample sizes explain this reversal. The posterior’s effective sample

Table 3: Simulation-based calibration of the posterior estimate (1000 trials; empirical coverage of central intervals at the nominal levels; Monte Carlo standard error ± 0.007 – 0.016). Nominal is best.

Target	Method	0.50	0.80	0.90	0.95
Gaussian mean (1-D)	asynchronous	0.51	0.80	0.90	0.94
	pyABC	0.35	0.63	0.75	0.83
Bimodal $\mathcal{N}(\theta^2, \sigma^2)$ (1-D)	asynchronous	0.50	0.82	0.91	0.96
	pyABC	0.45	0.82	0.90	0.95
g-and-k (4-D), A	asynchronous, full history	0.62	0.91	0.96	0.99
	pyABC	0.54	0.82	0.91	0.95
g-and-k (4-D), B	asynchronous, full history	0.54	0.83	0.91	0.96
	pyABC	0.45	0.74	0.85	0.90
g-and-k (4-D), g	asynchronous, full history	0.58	0.88	0.95	0.98
	pyABC	0.51	0.82	0.91	0.95
g-and-k (4-D), k	asynchronous, full history	0.56	0.83	0.92	0.96
	pyABC	0.47	0.74	0.86	0.92
g-and-k (4-D), A	asynchronous, top- k archive	—	—	0.85	0.89

Table 4: Cellular Potts 50^3 , five replicates per adaptive method and one rejection run, 3600s on 48 workers (means $\pm$ s.d. for contraction, medians otherwise). The first row is the asynchronous sampler at the benchmark’s original starting bandwidth (10, twenty times the prior-predictive median discrepancy), the rest at the production value (0.1). Rejection ABC is one run of 13,000 prior draws on all 48 workers, keeping the best 100. Contraction is $1 - \text{sd}/\text{sd}_{\text{prior}}$; the truth lies inside the central 90% interval of every run.

	sims	util.	$\epsilon_{(100)}$, end	contraction		ESS
				division rate	cell volume	
Asynchronous, $\epsilon_0 = 10$	13,300	99.8%	6.3×10^{-5}	$84 \pm 6\%$	$13 \pm 14\%$	645
Asynchronous, $\epsilon_0 = 0.1$	12,900	99.8%	7.6×10^{-5}	$92 \pm 1\%$	$81 \pm 1\%$	303
pyABC, $\epsilon_0 = 0.1$	5,400	42.8%	3.1×10^{-4}	$91 \pm 1\%$	$72 \pm 4\%$	66
Rejection ABC, best 100 of 13,000	13,000	—	2.9×10^{-3}	91%	67%	100

size stays at 238–296 at every checkpoint of the Gaussian run while the history grows from 10^5 to 1.3×10^6 particles. A controlled sweep puts ESS/k between 1.9 and 3.6 for $k \in \{50, 100, 200\}$. The estimate is reported at the tightest bandwidth reached, which is chosen by ESS retention against the archive (§3.3). Its effective support therefore stays at a few multiples of k however many particles are evaluated. In one dimension that support is small and stable; in four it is small and erratic (7 to 2987 across checkpoints and replicates), and accuracy follows it. The four-dimensional deficit is consistent with intermittent weight degeneracy rather than bias. The limitation appears once the target is identified well enough for the bandwidth to tighten.

A known truth on the real simulator. On the two-parameter Cellular Potts setup (one synthetic dataset with a known truth), all three methods recover the truth (Fig. 6, Table 4). The division rate is well identified by every method (91–92% contraction against the prior). For cell volume, contraction is $81\% \pm 1\%$ for the asynchronous estimator, $72\% \pm 4\%$ for pyABC’s last generation with its population weights, and 67% for the best 100 rejection draws. The truth lies inside the central 90% interval of every run (five per adaptive method, one rejection run); on a single dataset that is containment, not coverage. The asynchronous estimator is the tightest and has the largest effective sample (303 against pyABC’s 66 of 100, medians).

The starting bandwidth. The first row of Table 4 is the same sampler started at the benchmark’s original $\epsilon_0 = 10$, twenty times the prior-predictive median discrepancy of 0.46. It reports $13\% \pm 14\%$ contraction on the cell volume, erratic as well as poor, although its draws are as concentrated as the production run’s (the end tolerances agree within 20%). The runs differ in the bandwidth used for reporting. After the run’s 62 calls per worker, the schedule had not moved from ϵ_0 , which ended $321\times$ above the tolerance the draws supported, against $2\times$ at $\epsilon_0 = 0.1$. The transient is negligible on a simulator that affords millions of

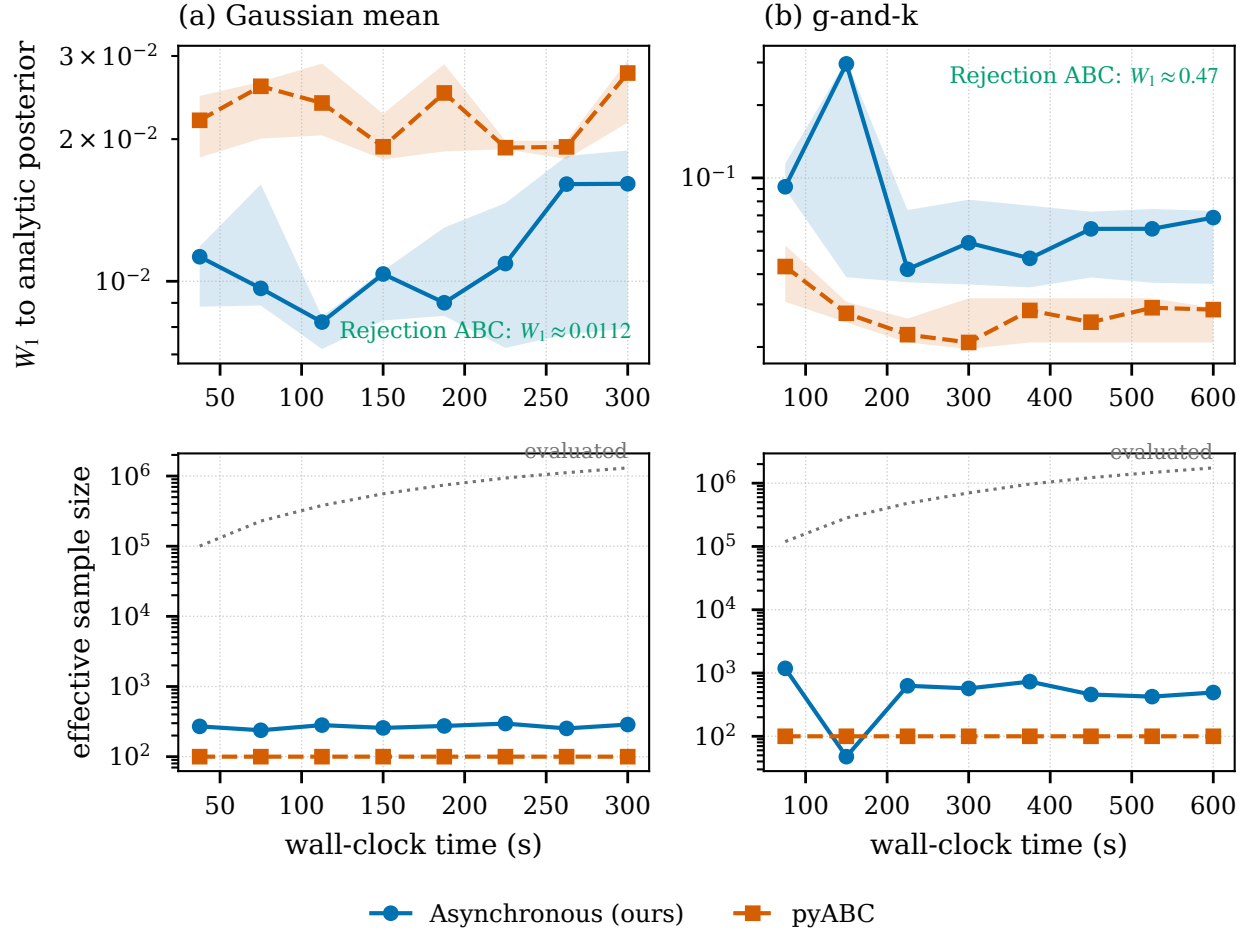


Figure 5: Each method’s own posterior (§5.3) scored against the analytic posterior (a) and the g-and-k reference posterior (b) over wall-clock prefixes. Top: W_1 , median and inter-quartile range over five replicates. Bottom: the effective sample size of the posterior estimate, with the evaluated-particle count dotted.

evaluations, which is why the cheap runs never showed it. It remains a binding limitation in the regime the method is designed for. The rule of §5.3, about a fifth of the prior-predictive median, recovers the posterior. Re-reporting a stored history at the k -th order statistic of its own discrepancies costs no simulations and reaches a comparable posterior (93%/85% on replicate 0); it is the check we apply. The barrierized twin shows the same transient on the cheap straggler benchmark (Appendix F.2).

The archive size. The effective sample size is a few multiples of k , but raising k has costs. Re-reporting one history at the order statistic for k from 10 to 1000 loosens the final bandwidth and lowers the cell-volume contraction from 85% to 78%. Running with $k = 300$ leaves the schedule at ϵ_0 for the whole hour. On the cheap benchmarks, $k = 100$ is Pareto-optimal for throughput and calibration (Appendix F). The archive size is a benchmark-dependent choice, made by checking calibration and the bandwidth reached.

7 Discussion

Against pyABC with the same kernel and schedule, the method is ahead on the tissue simulator in simulations per hour, tolerance at equal wall clock, contraction on the weakly identified parameter and effective sample size (Table 4); on the synthetic targets it is calibrated wherever pyABC under-covers, and it loses only where a simulation costs less than a few milliseconds. The results give a practitioner four rules.

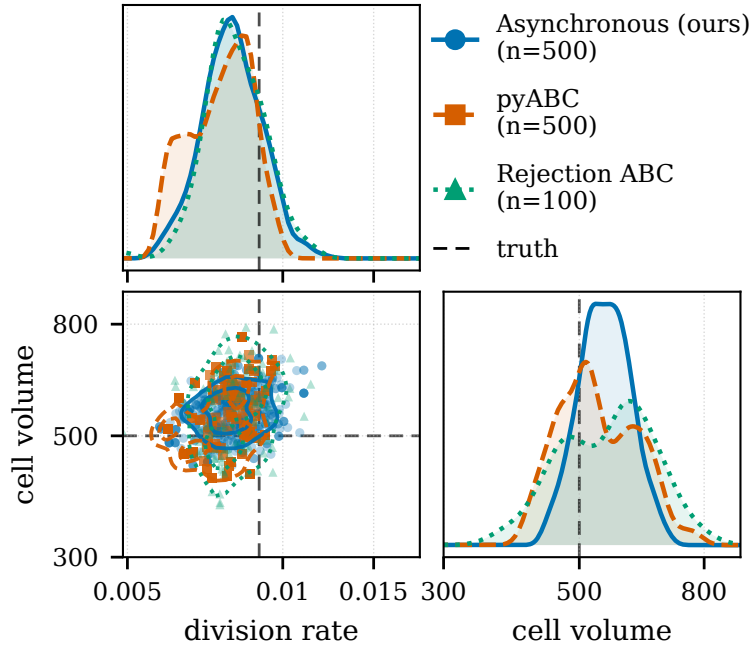


Figure 6: Cellular Potts posterior (division rate, cell volume) on the fixed production run, replicate 0: the asynchronous AMIS estimator (500 resampled draws), pyABC’s last completed generation with its population weights (500 resampled draws), and the 100 best rejection draws (Table 4). Axes are the log-uniform prior coordinates ticked in physical units; dashed lines mark the truth.

Use it above a few milliseconds per simulation. Below that the per-arrival coordination costs more than a barrier does, and a generation-based sampler with a population sized to the machine is the better tool; above it, removing the barrier recovers a cost that grows with the worker count and with the spread of simulation times (§6.3).

Predict the gain before paying for it. The barrier’s cost is computable from the asynchronous run’s own per-evaluation timing, within the domain stated in §5.3 (§6.1).

Set the starting bandwidth from the prior-predictive discrepancy scale. On an expensive simulator the whole run can be spent inside the bandwidth transient; re-reporting a stored history at the order statistic of its own discrepancies is the check (§6.4). A floor at $10^{-4}\epsilon_0$ puts the run under the limits of §4 and never binds on the g-and-k and Cellular Potts runs.

Keep the archive small and check calibration. Enlarging the archive loosens the final bandwidth and lengthens the transient, and the calibration checks are cheap to repeat (§6.4).

The effective-sample-size ceiling is where the method can still improve. Barrier removal delivers simulations predictably; converting all of them into posterior accuracy is limited by the reporting rule, which we adopted for its simplicity: report at the tightest bandwidth reached, chosen by ESS retention against the archive. Further work on this method belongs in that rule rather than in the scheduler.

8 Limitations

The limit statements hold up to the fidelity ratio r , of which two factors are measured and two bounded, and only on runs whose reporting bandwidth stays away from zero. A floor of $10^{-4}\epsilon_0$ guarantees this; it would have clipped the two near-instantaneous benchmarks but none of the g-and-k and Cellular Potts runs. Even where the limits apply, they describe the run’s continuation, since no reported run reached its floor (§4). Keeping every evaluated particle over-represents parameter regions that simulate quickly, the length bias of

deadline-limited sampling (Murray et al., 2021). Under the execution model the direct part of that effect is confined to the at most W candidates in flight at the deadline, an $O(W/n)$ perturbation (Proposition 9); the rest acts through the proposal sequence, which the weighting corrects up to r . A runtime-coupled study found no measurable shift of the posterior mean at the couplings tested, and we do not correct for the $O(W/n)$ term (Appendix F).

9 Conclusion

A generation barrier costs the ratio of the barrier-free throughput to the throughput a generation allows, and this cost can be predicted from the asynchronous run’s timing alone. The prediction matched a barrierized twin of our sampler on synthetic runtime laws and on a real tissue simulator. The generation-free sampler removes the barrier and, wherever a simulation costs more than a few milliseconds, converts the simulations it gains into a tighter tolerance. Its estimator targets the smooth-ABC posterior up to a partly measured fidelity ratio, is calibrated in one dimension and conservative in four, and is replayable from the log. What it delivers is bounded by the starting bandwidth and the archive size, and §7 says how to set them.

Broader Impact Statement

This work is methodology for likelihood-free inference on parallel machines. Its direct effect is on compute: it removes the idle time a generation barrier imposes on heterogeneous simulator workloads, and the prediction of §6.1 lets a practitioner estimate that saving from their own timing data before spending an allocation. We see no risk specific to the method beyond those of Bayesian inference in general.

Reproducibility

The repository accompanying this paper contains the summary data behind every figure and table, the scripts that regenerate each from the campaign output, and the committed output of every diagnostic quoted in the text, so every figure, table and quoted number can be regenerated from the repository alone. The full implementation (the Propulate propagator, the matched-kernel pyABC baseline, the analysis and plotting scripts, and the test suite), the raw per-particle records (≈ 30 GB for the Gaussian-mean campaign alone), the per-rank timing logs and the resolved run configurations will be released publicly upon publication.

References

- Emad Alamoudi, Felipe Reck, Nils Bundgaard, Frederik Graw, Lutz Brusch, Jan Hasenauer, and Yannik Schälte. A wall-time minimizing parallelization strategy for approximate Bayesian computation. *PLOS ONE*, 19(2):e0294015, 2024. doi: 10.1371/journal.pone.0294015.
- Mark A. Beaumont. Approximate Bayesian computation. *Annual Review of Statistics and Its Application*, 6:379–403, 2019. doi: 10.1146/annurev-statistics-030718-105212.
- Mark A. Beaumont, Jean-Marie Cornuet, Jean-Michel Marin, and Christian P. Robert. Adaptive approximate Bayesian computation. *Biometrika*, 96(4):983–990, 2009. doi: 10.1093/biomet/asp052.
- Marco Berghoff, Ivan Kondov, and Johannes Hötzer. Massively parallel stencil code solver with autonomous adaptive block distribution. *IEEE Transactions on Parallel and Distributed Systems*, 29(10):2282–2296, 2018. doi: 10.1109/TPDS.2018.2819672.
- Marco Berghoff, Jakob Rosenbauer, Felix Hoffmann, and Alexander Schug. Cells in Silico – introducing a high-performance framework for large-scale tissue modeling. *BMC Bioinformatics*, 21:436, 2020. doi: 10.1186/s12859-020-03728-7.
- Nicolas Bonneel, Julien Rabin, Gabriel Peyré, and Hanspeter Pfister. Sliced and Radon Wasserstein barycenters of measures. *Journal of Mathematical Imaging and Vision*, 51(1):22–45, 2015. doi: 10.1007/s10851-014-0506-3.

- Mónica F. Bugallo, Víctor Elvira, Luca Martino, David Luengo, Joaquín Míguez, and Petar M. Djurić. Adaptive importance sampling: The past, the present, and the future. *IEEE Signal Processing Magazine*, 34(4):60–79, 2017. doi: 10.1109/MSP.2017.2699226.
- Olivier Cappé, Arnaud Guillin, Jean-Michel Marin, and Christian P. Robert. Population Monte Carlo. *Journal of Computational and Graphical Statistics*, 13(4):907–929, 2004. doi: 10.1198/106186004X12803.
- Jianmin Chen, Xinghao Pan, Rajat Monga, Samy Bengio, and Rafal Jozefowicz. Revisiting distributed synchronous SGD. *arXiv preprint arXiv:1604.00981*, 2016.
- Jean-Marie Cornuet, Jean-Michel Marin, Antonietta Mira, and Christian P. Robert. Adaptive multiple importance sampling. *Scandinavian Journal of Statistics*, 39(4):798–812, 2012. doi: 10.1111/j.1467-9469.2011.00756.x.
- Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020. doi: 10.1073/pnas.1912789117.
- Jeffrey Dean, Greg S. Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc’Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, and Andrew Y. Ng. Large scale distributed deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.
- Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. An adaptive sequential Monte Carlo method for approximate Bayesian computation. *Statistics and Computing*, 22(5):1009–1020, 2012. doi: 10.1007/s11222-011-9271-y.
- Christopher C. Drovandi and Anthony N. Pettitt. Estimation of parameters for macroparasite population evolution using approximate Bayesian computation. *Biometrics*, 67(1):225–233, 2011. doi: 10.1111/j.1541-0420.2010.01410.x.
- Ritabrata Dutta, Marcel Schoengens, Lorenzo Pacchiardi, Avinash Ummadisingu, Nicole Widmer, Pierre Künzli, Jukka-Pekka Onnela, and Antonietta Mira. ABCpy: A high-performance computing perspective to approximate Bayesian computation. *Journal of Statistical Software*, 100(7):1–38, 2021. doi: 10.18637/jss.v100.i07.
- Paul Fearnhead and Dennis Prangle. Constructing summary statistics for approximate Bayesian computation: semi-automatic approximate Bayesian computation. *Journal of the Royal Statistical Society: Series B*, 74(3):419–474, 2012. doi: 10.1111/j.1467-9868.2011.01010.x.
- Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021.
- Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.
- François Graner and James A. Glazier. Simulation of biological cell sorting using a two-dimensional extended Potts model. *Physical Review Letters*, 69(13):2013–2016, 1992. doi: 10.1103/PhysRevLett.69.2013.
- David Greenberg, Marcel Nonnenmacher, and Jakob Macke. Automatic posterior transformation for likelihood-free inference. In *International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2404–2414, 2019.
- Tsuyoshi Hirashima, Elisabeth G. Rens, and Roeland M. H. Merks. Cellular Potts modeling of complex multicellular behaviors in tissue morphogenesis. *Development, Growth & Differentiation*, 59(5):329–339, 2017.
- Emanuel Klinger, David Rickert, and Jan Hasenauer. pyABC: distributed, likelihood-free inference. *Bioinformatics*, 34(20):3591–3593, 2018. doi: 10.1093/bioinformatics/bty361.

- Jochen Kursawe, Ruth E. Baker, and Alexander G. Fletcher. Approximate Bayesian computation reveals the importance of repeated measurements for parameterising cell-based models of growing tissues. *Journal of Theoretical Biology*, 443:66–81, 2018.
- Ben Lambert, Adam L. MacLean, Alexander G. Fletcher, Alexander N. Combes, Melissa H. Little, and Helen M. Byrne. Bayesian inference of agent-based models: a tool for studying kidney branching morphogenesis. *Journal of Mathematical Biology*, 76:1673–1697, 2018.
- Maxime Lenormand, Franck Jabot, and Guillaume Deffuant. Adaptive approximate Bayesian computation for complex models. *Computational Statistics*, 28(6):2777–2796, 2013. doi: 10.1007/s00180-013-0428-3.
- Jarno Lintusaari, Henri Vuollekoski, Antti Kangasrääsiö, Kusti Skytén, Marko Järvenpää, Pekka Marttinen, Michael U. Gutmann, Aki Vehtari, Jukka Corander, and Samuel Kaski. ELFI: Engine for likelihood-free inference. *Journal of Machine Learning Research*, 19(16):1–7, 2018.
- Jan-Matthis Lueckmann, Jan Boelts, David Greenberg, Pedro Gonçalves, and Jakob Macke. Benchmarking simulation-based inference. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 130 of *Proceedings of Machine Learning Research*, pp. 343–351, 2021.
- Jean-Michel Marin, Pierre Pudlo, and Mohammed Sedki. Consistency of adaptive importance sampling and recycling schemes. *Bernoulli*, 25(3):1977–1998, 2019. doi: 10.3150/18-BEJ1042.
- Lawrence M. Murray, Anthony Lee, and Pierre E. Jacob. Parallel resampling in the particle filter. *Journal of Computational and Graphical Statistics*, 25(3):789–805, 2016. doi: 10.1080/10618600.2015.1062015.
- Lawrence M. Murray, Sumeetpal S. Singh, and Anthony Lee. Anytime Monte Carlo. *Data-Centric Engineering*, 2:e7, 2021. doi: 10.1017/dce.2021.6.
- Art Owen and Yi Zhou. Safe and effective importance sampling. *Journal of the American Statistical Association*, 95(449):135–143, 2000. doi: 10.1080/01621459.2000.10473909.
- Brooks Paige, Frank Wood, Arnaud Doucet, and Yee Whye Teh. Asynchronous anytime sequential Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.
- George Papamakarios and Iain Murray. Fast ϵ -free inference of simulation models with Bayesian conditional density estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.
- G. D. Rayner and H. L. MacGillivray. Numerical maximum likelihood estimation for the g-and-k and generalized g-and-h distributions. *Statistics and Computing*, 12(1):57–75, 2002. doi: 10.1023/A:1013120305780.
- Benjamin Recht, Christopher Ré, Stephen J. Wright, and Feng Niu. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, 2011.
- Yannik Schälte, Emanuel Klinger, Paul Stapor, and Jan Hasenauer. pyABC 0.12.3: high-performance distributed, likelihood-free inference. *Bioinformatics*, 38(13):3354–3355, 2022. doi: 10.1093/bioinformatics/btac321.
- Scott A. Sisson, Yanan Fan, and Mark M. Tanaka. Sequential Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 104(6):1760–1765, 2007. doi: 10.1073/pnas.0607208104.
- Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration. *arXiv preprint arXiv:1804.06788*, 2018.
- Oskar Taubert, Marie Weißer, Daniel Kelle, Markus Bayer, Hilmar Knüttel, Claudia Comito, Achim Streit, and Markus Götz. Massively parallel genetic optimization through asynchronous propagation of populations. In *International Conference on High Performance Computing (ISC High Performance)*, pp. 106–124, 2023. doi: 10.1007/978-3-031-32041-5_6.

Tina Toni, David Welch, Natalja Strelkowa, Anders Ipsen, and Michael P. H. Stumpf. Approximate Bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface*, 6(31):187–202, 2009. doi: 10.1098/rsif.2008.0172.

Eric Veach and Leonidas J. Guibas. Optimally combining sampling techniques for Monte Carlo rendering. In *Proceedings of SIGGRAPH '95*, pp. 419–428, 1995. doi: 10.1145/218380.218498.

Christelle Vergé, Cyrille Dubarry, Pierre Del Moral, and Éric Moulines. On parallel implementation of sequential Monte Carlo methods: The island particle model. *Statistics and Computing*, 25(2):243–260, 2015. doi: 10.1007/s11222-013-9429-x.

Richard D. Wilkinson. Approximate Bayesian computation (ABC) gives exact results under the assumption of model error. *Statistical Applications in Genetics and Molecular Biology*, 12(2):129–141, 2013. doi: 10.1515/sagmb-2013-0010.

A The fidelity ratio

This appendix relates the analysis to existing AMIS theory, bounds how far the tilt by r can move the target, and decomposes r and measures part of it on stored runs.

Relation to existing AMIS theory. For a fixed proposal, the r -tilted limit of Corollary 2 is the elementary observation that self-normalized importance sampling against a mismatched denominator converges to a correspondingly tilted target. The present scheme, however, is adaptive at every draw, so the argument runs through a martingale array rather than an i.i.d. one. The available AMIS results do not cover it: the convergence argument of Cornuet et al. (2012) is heuristic, and Marin et al. (2019) restrict the adaptation so that the weighting stage decouples. The prior floor in the *denominator* bounds every weight by $1/\delta$ with no moment or density-ratio condition. This lets the method use a local archive proposal at all, for which the usual two-sided ratio bound fails.

Proposition 4 (how far the tilt can move the target). *Let $\zeta_0 := \pi_{\epsilon_\infty}(|r - 1|)$. Then*

$$\|\pi_{\epsilon_\infty}^r - \pi_{\epsilon_\infty}\|_{\text{TV}} = \frac{\pi_{\epsilon_\infty}(|r - \pi_{\epsilon_\infty}(r)|)}{2\pi_{\epsilon_\infty}(r)} \quad \text{and} \quad \zeta_0 < 1 \implies \|\pi_{\epsilon_\infty}^r - \pi_{\epsilon_\infty}\|_{\text{TV}} \leq \frac{\zeta_0}{1 - \zeta_0}. \quad (13)$$

If $|r - 1| \leq \eta_A$ on a set $A \subseteq \Theta$ and $|r - 1| \leq M$ off it, then $\zeta_0 \leq \zeta := \eta_A + M\pi_{\epsilon_\infty}(A^c)$, and the bound holds with ζ in place of ζ_0 whenever $\zeta < 1$.

The identity says that the tilt moves the target through the variation of r over posterior mass, not through its distance from one: $\pi_{\epsilon_\infty}^{cr} = \pi_{\epsilon_\infty}^r$ for every constant $c > 0$. The bound replaces the centered absolute moment by the uncentered ζ_0 and the divisor $\pi_{\epsilon_\infty}(r)$ by $1 - \zeta_0$, and is loose by exactly that much. Estimating ζ_0 from a stored run requires *untilting*: the normalized weights $\hat{w}_i$ target π^r , not π_{ϵ_∞} , so $\sum_i \hat{w}_i |r(\theta_i) - 1|$ estimates $\pi^r(|r - 1|)$; weights proportional to $W_{i,n}/r(\theta_i)$ target π_{ϵ_∞} and give $\hat{\zeta}_0$.

A.1 What r is for this implementation

We decompose r by inserting intermediate mixtures between the density actually sampled from and the function used in the denominator, changing one implementation feature at a time. Let $\bar{q}^{(1)} = \frac{1}{n} \sum_{i \leq n} q_i$ use the *nominal* archive proposals: the truncated mixtures the propagator intends, with exact normalizing constants and without the rejection cap or the underflow redraws. We obtain $\bar{q}^{(2)}$ by replacing those constants with the implementation’s product-of-marginals values, and $\bar{q}^{(3)}$ by taking the m -point draw-proportional version of $\bar{q}^{(2)}$ carrying the observed bootstrap share ν_n . Assume each of the four displayed ratios has a uniform limit, finite and bounded away from zero. Fidelity alone does not give this, since (9) constrains only their product. Writing $\bar{q}$ for the m -snapshot denominator (7),

$$r = \underbrace{\lim \frac{\bar{q}^*}{\bar{q}^{(1)}}}_{\text{(d) cap, redraws, asynchrony}} \cdot \underbrace{\lim \frac{\bar{q}^{(1)}}{\bar{q}^{(2)}}}_{\text{(c) normalizer}} \cdot \underbrace{\lim \frac{\bar{q}^{(2)}}{\bar{q}^{(3)}}}_{\text{(b) quadrature}} \cdot \underbrace{\lim \frac{\bar{q}^{(3)}}{\bar{q}}}_{\text{(a) floor}}.$$

The product telescopes for any positive intermediates, so the content is in the choice of intermediates: a different insertion order attributes the total differently, and the factors are a decomposition of one quantity rather than four independent errors.

(a) *The prior floor.* When genuine bootstrap draws carry less than a δ share of the history, $\lambda_n = \delta > \nu_n$ and the denominator holds more prior mass than the sample does. With $u = q_\infty/\pi$ the limiting proposal-to-prior ratio, this contributes $r_{\text{floor}} = (\nu_\infty + (1 - \nu_\infty)u)/(\delta + (1 - \delta)u)$, so $r_{\text{floor}} - 1 = (\delta - \nu_\infty)(u - 1)/(\delta + (1 - \delta)u)$: r_{floor} is exactly 1 when $\nu_\infty \geq \delta$, below 1 wherever the proposal is thinner than the prior, and $r_{\text{floor}} \rightarrow 1$ as $\delta \rightarrow 0$, i.e. as m grows. This is the one contribution available in closed form (Lemma 10).

(b) *The snapshot quadrature.* The m -point mixture stands in for the n -point one, with error at most V_n/m where $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ is the total variation of the *reconstructed* proposal path; this controls factor (b), not the fidelity of the whole. Fixing m therefore trades cost against fidelity; see the remark below.

(c) *The truncation constant.* Each mixture component is normalized by the product of its per-axis in-box masses, which is exact for a diagonal perturbation covariance and increasingly accurate as a component's mass sits away from the faces of the box relative to its own scale, but an approximation otherwise.

(d) *Capped rejection, underflow redraws, and (asynchronously) the arrival-order reconstruction.* A parent's perturbation is rejection-sampled into the box for at most 992 attempts, so component j , with mixture weight W_j and in-box mass Z_j , is realized with probability $W_j[1 - (1 - Z_j)^{992}]$ and the residual is emitted as a prior draw; and a candidate whose proposal-time denominator falls below 10^{-12} is redrawn up to five times, the last one being emitted regardless. The cap changes the law relative to unlimited rejection whenever $Z_j < 1$, through the residual probability $(1 - Z_j)^{992}$; $Z_j \approx 10^{-3}$ is the scale below which that residual becomes appreciable, not a threshold above which it vanishes. Under asynchronous execution, this factor additionally includes the difference between the proposal each candidate came from, formed on the prefix its worker held, and the snapshots the post-hoc pass rebuilds from an arrival-ordered log. The emitted density is the conditional law of the candidate (Lemma 5), so no further gap arises at that step. Prefixes of equal length in the two orders differ in at most W candidates, those in flight at the time; therefore, each rebuilt snapshot differs from the proposal in force at that index by a boundary of at most W particles.

Measured, in part. Two of the four factors, the prior floor and the snapshot quadrature, can be estimated rather than bounded, because $\hat{\pi}_n$ is a post-hoc pass: recomputing (7) at a much larger m and at the observed prior share gives a reference against which those factors are evaluated at every particle. The truncation constant and factor (d) are bounded rather than measured. The five Cellular Potts production replicates (Appendix E) have about 12,900 asynchronous evaluations each across 48 workers and two correlated dimensions, so that (c) is live. Both measured factors, (a) and (b), are active: the bootstrap draws make up 1.1% of each history against a floor of $\delta = 2.4\%$, so (a) contributes $\hat{\zeta} = 0.011$ and (b) 0.024–0.028, together 0.035–0.039 with $r \in [0.24, 1.41]$. The posterior estimate lies 0.2–0.5% in total variation from the reference at $m = 400$ and the observed prior share. Posterior means agree within 0.012 standard deviations and marginal widths within 1%, narrower in nine of the ten marginals, which is the tail-suppressing direction that (a) predicts; the effective sample size is unchanged. On a one-dimensional Gaussian-mean history, where the exact n -point mixture is computable, contributions (a) and (b) together give $\hat{\zeta} = 0.042$; the total-variation distance from the reference that removes them is 0.26%, and there too the estimate is 0.5% narrower. Of contribution (d), the two events a stored history records, a rejection fallback and an exhausted redraw, occurred neither in the 12,000 draws of that history nor in the 63,699 archive-phase draws of the five Cellular Potts replicates. A redraw that succeeded on a later attempt leaves no trace, so this is a necessary condition for the event, not a bound on its probability. The bound in (13) is loose against both direct measurements by an order of magnitude (3.6–4.0% against 0.2–0.5% on Cellular Potts, 4.4% against 0.26% on the one-dimensional history), as sup-norm-free bounds valid for every f generally are. The identity in (13) says what the direct measurements estimate: the centered absolute moment of the removed factors under the reference, divided by twice their mean.

Does the proposal path settle? Factor (b) is controlled by the total variation V_n of the reconstructed proposal path, which a stored history makes measurable. Rebuilding q_τ along the five Cellular Potts production replicates, the per-step drift $\|q_{\tau+1} - q_\tau\|_\infty$ decays as $\tau^{-0.7}$ to $\tau^{-0.95}$. The archive's membership changes

only 825–874 times in about 12,900 draws, tracking $k \log n$ at a ratio between 0.54 and 0.92 across the whole run. This is the rate a record-statistics heuristic predicts, since the k -th best of n draws improves with probability $\approx k/n$. The proposal path’s total variation is therefore $O(\log n)$, the regime in which a snapshot count growing faster than $\log n$ removes factor (b) (Remark 2). The measurement spans one benchmark over 1.3 decades of n per replicate, so it is a diagnostic rather than an asymptotic statement.

Fixed versus growing m . With m fixed, the m -point mixture tracks the n -point one only if the proposal path settles, the error being V_n/m . Letting $m = m_n \rightarrow \infty$ with $V_n/m_n \rightarrow 0$ removes *this* contribution to r (it does not give $r \equiv 1$, since the other three factors remain), and the proofs go through for any $m_n = o(n/\log n)$ *provided the floor δ is held fixed rather than tied to m* . The implementation ties them through $\delta = 0.5/(m+1)$, so growing m there would send $\delta \rightarrow 0$ and lose the uniform weight bound $1/\delta$ of Lemma 6, which fixes the constants in the law of large numbers. The post-hoc pass can decouple the two (Appendix B, Remark 2). The retroactive pass then costs $\mathcal{O}(nm_n k)$ instead of $\mathcal{O}(nk)$, and the literal cumulative mixture is the endpoint $m_n = n$ at $\mathcal{O}(n^2 k)$. On the Gaussian-mean history the posterior estimate is within 3×10^{-4} in total variation of the exact n -point mixture by $m = 50$, so a $2.5\times$ more expensive post-hoc pass removes factor (b) almost entirely. The paper reports the $m = 21$ end; §3.5 gives its cost.

B Proofs

B.1 Setting

We work with a filtration $(\mathcal{F}_i)$ for which, writing $\mathbb{E}_{i-1}[\cdot] = \mathbb{E}[\cdot \mid \mathcal{F}_{i-1}]$, the pair $\xi_i = (\theta_i, \rho_i)$ is $\mathcal{F}_i$ -measurable and

$$\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i, \quad \rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i). \quad (14)$$

Particles are indexed by *proposal* time, not arrival time. Under sequential execution $\mathcal{F}_i = \sigma(\xi_1, \dots, \xi_i)$ serves and (14) holds by construction. Under asynchronous execution a discrepancy ρ_i may still be in flight when candidate $i+1$ is proposed, so the σ -field of everything realized by that moment does not contain ξ_i , and a larger filtration is needed. A two-worker example shows the difficulty. Worker A proposes candidate 1, whose simulation is slow; worker B proposes candidate 2, receives its result, and proposes candidate 3 while candidate 1 is still running. When candidate 3 is proposed, ξ_1 has not been realized. Conditioning on ξ_1, ξ_2 instead does not recover the density worker B emitted either: B built it from ξ_2 alone, and which results had reached B depends on the durations of the two simulations, which ξ_1, ξ_2 do not record. The following lemma supplies a filtration from the execution model that adds the durations, the worker identities and the delivery randomness, and under which the emitted density is the conditional law of the candidate.

Lemma 5 (adapted filtration under the execution model). *Let Assumption 4 hold, let w_i be the worker that proposes candidate i (in this lemma and its proof w_i is a worker index, not the parent weight of the history) and D_i the duration of its simulation, and let $\mathcal{F}_i := \sigma((\theta_j, \rho_j, D_j, w_j)_{j \leq i}, \mathcal{E})$, where $\mathcal{E}$ is the σ -field of the exogenous scheduling randomness of (B3). Then ξ_i is $\mathcal{F}_i$ -measurable and (14) holds with $\tilde{q}_i$ the proposal density that worker w_i emits on the prefix of the history it has received. Moreover, for every τ the arrival-ordered prefix of length τ and the proposal-ordered prefix $\{1, \dots, \tau\}$ differ in at most W candidates each.*

Proof. Candidate i is proposed when a worker becomes free, so its proposal time T_i is the completion time $T_j + D_j$ of an earlier candidate $j < i$ of the same worker (or a start time, for the first W candidates). By (B3) and induction on i , the time T_i , the worker w_i and the set $A_i \subset \{1, \dots, i-1\}$ of candidates whose results have reached w_i by T_i are functions of $(T_j, D_j, w_j)_{j < i}$ and of $\mathcal{E}$. They are therefore $\mathcal{F}_{i-1}$ -measurable. The emitted density q_i^{em} is a function of the history restricted to A_i , so it is $\mathcal{F}_{i-1}$ -measurable too. By (B2), θ_i is drawn from q_i^{em} with randomness independent of $\mathcal{F}_{i-1}$, so the conditional law of θ_i given $\mathcal{F}_{i-1}$ is q_i^{em} : the first part of (14), with $\tilde{q}_i = q_i^{\text{em}}$. By (B1) and (B3), (ρ_i, D_i) given θ_i is independent of $(\theta_j, \rho_j, D_j, w_j)_{j < i}$ and of $\mathcal{E}$, which generate $\mathcal{F}_{i-1}$; hence $\rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i)$. For the last claim, let P be the set of candidates proposed before the τ -th arrival and A the set of the first τ arrivals. Then $A \subseteq P$, $\{1, \dots, \tau\} \subseteq P$ since $|P| \geq \tau$, and $P \setminus A$ is the set in flight at that moment, of size at most W ; so $\{1, \dots, \tau\} \setminus A \subseteq P \setminus A$ has at most W elements, and $A \setminus \{1, \dots, \tau\}$ has the same number. $\square$

Completion times steer which worker proposes next and which prefix it sees. The lemma says that this steering changes *which* proposal density is used, a predictable choice, and not the law of the candidate given that density, which is fixed by fresh randomness. The post-hoc estimator consumes a rank's *arrival*-ordered log and reconstructs the q_{τ_s} from its prefixes. By the last clause of the lemma, each reconstructed snapshot differs from the proposal in force at that index by a boundary of at most W particles. This boundary is the asynchrony contribution to r (§A.1(d)). The lemma fails if simulator outputs are coupled across calls (a shared seed, say) or if the sampler's stream is correlated with runtimes; both are checkable implementation properties, and Table 5 records them for ours.

No structural form is imposed on $\tilde{q}_i$: it is whatever the archive rule, the capped rejection, the underflow redraws and the bootstrap phase jointly produce, and it enters the proofs only through $\tilde{q}_n^* = \frac{1}{n} \sum_{i \leq n} \tilde{q}_i$ of (8), for which the denominator $\tilde{q}_n$ of (7) is an m -component stand-in. Let $\mathcal{Q}$ be the set of positive functions of the form (7) whose components satisfy Assumption 2; nothing below requires an element of $\mathcal{Q}$ to integrate to one. Elements of $\mathcal{Q}$ are parameterized by $p_q = (\text{means, component weights, covariances, } \alpha, \lambda)$ ranging over a compact set $P_q \subset \mathbb{R}^D$, $D = m(kd + k + d(d+1)/2) + m + 1$, the last coordinate being the prior share $\lambda \in [\delta, 1]$ of (7), on which $\tilde{q}_n$ depends affinely.

Lemma 6 (bounded weights). *Under Assumptions 2–3, $0 \leq W_{i,n} \leq 1/\delta$ for all $i \leq n$ and all n , surely, whatever the sampler does.*

Proof. All components of (7) are non-negative, so $\tilde{q}_n \geq \delta\pi$ pointwise on Θ . Hence $W_{i,n} = \pi(\theta_i)K_{\epsilon_n}(\rho_i)/\tilde{q}_n(\theta_i) \leq K_{\epsilon_n}(\rho_i)/\delta \leq 1/\delta$, the last step by $K_{\epsilon} \leq 1$ (Assumption 1). The prior cancels exactly, so the bound needs neither a density-ratio condition nor a moment condition. $\square$

Lemma 7 (uniform SLLN for bounded martingale differences). *Let $P \subset \mathbb{R}^D$ be compact and $\{\psi_p\}_{p \in P}$ measurable functions on a measurable space $\mathcal{X}$ with $\sup_p \|\psi_p\|_{\infty} \leq B$ and $|\psi_p(x) - \psi_{p'}(x)| \leq \Lambda\|p - p'\|$ for every $x \in \mathcal{X}$. Let (ξ_i) be $\mathcal{X}$ -valued with ξ_i $\mathcal{F}_i$ -measurable. Then*

$$\sup_{p \in P} \left| \frac{1}{n} \sum_{i \leq n} (\psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)) \right| \longrightarrow 0 \quad \text{a.s.}$$

Proof. Fix p . The variables $D_i^p := \psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)$ form a martingale difference sequence with $|D_i^p| \leq 2B$, so by the Azuma–Hoeffding inequality $\mathbb{P}(|\sum_{i \leq n} D_i^p| \geq nt) \leq 2\exp(-nt^2/8B^2)$ for every $t > 0$. Fix $\eta > 0$ and let P_{η} be a finite η -net of P , of cardinality at most $C_P\eta^{-D}$. A union bound gives $\mathbb{P}(\max_{p \in P_{\eta}} |\sum_{i \leq n} D_i^p| \geq nt) \leq 2C_P\eta^{-D} \exp(-nt^2/8B^2)$, which is summable in n (the net is fixed, so its cardinality does not grow with n); by Borel–Cantelli, $\limsup_n \max_{p \in P_{\eta}} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t$ a.s. For arbitrary $p \in P$ pick $p' \in P_{\eta}$ with $\|p - p'\| \leq \eta$; then $|D_i^p - D_i^{p'}| \leq 2\Lambda\eta$ by the Lipschitz hypothesis (which also bounds the conditional expectations), so $\limsup_n \sup_{p \in P} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t + 2\Lambda\eta$ a.s. Intersecting the almost-sure events over the countable family $t = 1/r$, $\eta = 1/j$ with $r, j \in \mathbb{N}$ (here r is an integer index, not the fidelity ratio), gives the claim. $\square$

Lemma 8 (conditional-mean identity). *Let $\phi : \Theta \rightarrow \mathbb{R}$ be bounded measurable, and let the bandwidth $\epsilon \in (0, \epsilon_0]$ and the positive function $q \geq \delta\pi$ each be either deterministic or $\mathcal{F}_{i-1}$ -measurable. Then under (14),*

$$\mathbb{E}_{i-1} \left[\frac{\phi(\theta_i) K_{\epsilon}(\rho_i)}{q(\theta_i)} \right] = \int_{\Theta} \phi L_{\epsilon} \frac{\tilde{q}_i}{q} d\theta.$$

Proof. Condition on $\mathcal{F}_{i-1}$, given which ϵ and q are constants, and integrate first over $\rho_i \sim p(\cdot | \theta_i)$, which gives the factor $L_{\epsilon}(\theta_i) = \mathbb{E}[K_{\epsilon}(\rho) | \theta_i]$, then over $\theta_i \sim \tilde{q}_i$. Fubini applies because the integrand is bounded by $\|\phi\|_{\infty}/(\delta\pi_-)$. An $\mathcal{F}_{\infty}$ -measurable q , such as the run's final denominator $\tilde{q}_n$, is *not* admissible here; the proof below handles it through uniformity instead. The case $\phi = \pi h$ is the one used below. $\square$

The class used below. Take $\mathcal{X} = \Theta \times [0, \infty)$, $\xi_i = (\theta_i, \rho_i)$, and, for a fixed bounded measurable h ,

$$\psi_{(\epsilon, p_q)}(\theta, \rho) := \frac{\pi(\theta) h(\theta) K_\epsilon(\rho)}{q_{p_q}(\theta)}, \quad (\epsilon, p_q) \in P_a := [a, \epsilon_0] \times P_q, \quad a \in (0, \epsilon_0]. \quad (15)$$

Each P_a is compact and *deterministic*, which matters: Lemma 7 cannot be applied over a parameter set that depends on the run, and $[\epsilon_\infty, \epsilon_0]$ would be such a set when ϵ_∞ is random. We therefore apply the lemma on $P_{1/j}$ for every $j \in \mathbb{N}$ with $1/j \leq \epsilon_0$ and intersect the countably many almost-sure events; since $\epsilon_n \downarrow \epsilon_\infty > 0$ a.s., each path has $\epsilon_n \in [1/j, \epsilon_0]$ for all n once $1/j \leq \epsilon_\infty$, so the resulting uniform bound covers $(\epsilon_n, \bar{q}_n)$ on that path. No deterministic bandwidth floor is required. On P_a , $\sup \|\psi\|_\infty \leq \|h\|_\infty / \delta$ by the computation of Lemma 6. For the Lipschitz hypothesis: each truncated component density $\varphi(\theta; \mu_j, \Sigma) / Z(\mu_j, \Sigma)$ has parameter-derivatives bounded uniformly over $\theta \in \Theta$, because Θ is compact, $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ bounds φ and its derivatives, and the truncation mass is bounded below, $Z(\mu_j, \Sigma) \geq c_0 > 0$, uniformly for $\mu_j \in \Theta$. The implementation's constant is the *product of per-axis* in-box masses, which is where the box form of Θ in Assumption 2 enters. Each factor is $\Phi((\text{hi}_d - \mu_{jd})/\sigma_d) - \Phi((\text{lo}_d - \mu_{jd})/\sigma_d)$. Since μ_{jd} lies in $[\text{lo}_d, \text{hi}_d]$, that interval contains a half-width $w_d/2$ on one side of μ_{jd} , giving a factor at least $\Phi(w_d/2\sigma_+) - \frac{1}{2} > 0$. The product over coordinates, $\prod_d (\Phi(w_d/2\sigma_+) - \frac{1}{2})$, is therefore bounded below by a constant depending only on the box and σ_+ and is smooth in (μ_j, Σ) with bounded derivatives there. (The exact joint mass is likewise bounded below, as a positive continuous function of (μ_j, Σ) on a compact set, so nothing in the argument depends on which normalizer is used; the box is needed to define the implementation's constant, not to bound it.) Hence every $q \in \mathcal{Q}$ (a set of positive *functions*, not necessarily normalized; Assumption 2) obeys $\delta\pi_- \leq q \leq q_+ < \infty$ and $q \mapsto q(\theta)$ is Lipschitz in p_q uniformly in θ , in the prior share λ affinely with coefficients bounded by π_+ and q_+ ; so is $1/q$, since q is bounded away from 0. With K_ϵ Lipschitz in ϵ (Assumption 1) and πh bounded, (15) satisfies the hypotheses of Lemma 7. Note that on $\{1/j \leq \epsilon_\infty\}$ we have $(\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n , and that $\bar{q}_\infty \geq \delta\pi$ by taking uniform limits.

B.2 Proof of Theorem 1 and Corollaries 2–3

The prior floor bounds every weight (Lemma 6). For a fixed bandwidth and denominator, the weighted average minus its predictable compensator is an average of bounded martingale differences and vanishes (Lemma 7). The class of bandwidths and denominators is compact and the weights are Lipschitz in its parameters, so this convergence is uniform over the class. Uniformity allows the run-dependent pair $(\epsilon_n, \bar{q}_n)$ to be substituted, after which Lemma 8 identifies the compensator as the tilted integral.

Work on the event $\{1/j \leq \epsilon_\infty\}$ for a fixed j , so that $\hat{p}_n := (\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n ; the conclusions then follow on the union over j , which is $\{\epsilon_\infty > 0\}$. Fix a bounded measurable h and define, for each *fixed* $p \in P_{1/j}$, $\Gamma_i(p) := \mathbb{E}_{i-1}[\psi_p(\xi_i)]$ and $\bar{\Gamma}_n(p) := \frac{1}{n} \sum_{i \leq n} \Gamma_i(p)$. On the almost-sure event of Lemma 7 for $P_{1/j}$ the bound holds simultaneously for all $p \in P_{1/j}$, so the *function* $\bar{\Gamma}_n$ may be evaluated at the random $\hat{p}_n$:

$$\frac{1}{n} \sum_{i \leq n} W_{i,n} h(\theta_i) = \bar{\Gamma}_n(\hat{p}_n) + o(1) \stackrel{\text{Lem. 8}}{=} \int_{\Theta} \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} d\theta + o(1) \quad \text{a.s.}, \quad (16)$$

using $\frac{1}{n} \sum_{i \leq n} \tilde{q}_i = \bar{q}_n^*$. This is (10). The middle term is the function $\bar{\Gamma}_n(\cdot)$ evaluated at $\hat{p}_n$; it must not be read as $\mathbb{E}_{i-1}[\psi_{\hat{p}_n}(\xi_i)]$. The reason is that $\hat{p}_n$ depends on draws after i and is not $\mathcal{F}_{i-1}$ -measurable, so Lemma 7 does not control that conditional expectation. For (11), apply (10) with $h = f$ and with $h \equiv 1$, and write $A_n = a_n + o(1)$ and $B_n = b_n + o(1)$, with A_n, B_n the two weighted averages and a_n, b_n the two integrals. If $\liminf_n B_n > 0$ then $b_n \geq c > 0$ for all large n , and $|A_n/B_n - a_n/b_n| \leq |A_n - a_n|/B_n + |a_n/b_n| |b_n - B_n|/B_n \rightarrow 0$, since $|a_n/b_n| \leq \|f\|_\infty$. This proves the theorem.

Corollary 2. Split the integral in (10) against $\int \pi h L_{\epsilon_\infty} r$:

$$\left| \int \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} - \int \pi h L_{\epsilon_\infty} r \right| \leq \|h\|_\infty \pi_+ |\Theta| \left[R L_K(1/j) |\epsilon_n - \epsilon_\infty| + \left\| \frac{\bar{q}_n^*}{\bar{q}_n} - r \right\|_\infty \right],$$

on the event $\{1/j \leq \epsilon_\infty\}$ of the localization above, using $|L_{\epsilon_n} - L_{\epsilon_\infty}| \leq L_K(1/j) |\epsilon_n - \epsilon_\infty|$ (Assumption 1 and Jensen; every ϵ_n and ϵ_∞ lies in $[1/j, \epsilon_0]$ there) and $L_{\epsilon_\infty} \leq 1$. Here $R = r_+ + 1$ bounds $\bar{q}_n^*/\bar{q}_n$ for all large

n by (9), with no structural bound on $\tilde{q}_i$ needed. Both bracketed terms vanish, the first because $\epsilon_n \rightarrow \epsilon_\infty$ and the second by (9). Hence $\frac{1}{n} \sum_i W_{i,n} h(\theta_i) \rightarrow \int \pi h L_{\epsilon_\infty} r$ a.s. on the event. Applying this with $h = f$ and with $h \equiv 1$ and dividing, which is legitimate because $Z_r = \int \pi L_{\epsilon_\infty} r \geq r_- Z_{\epsilon_\infty} > 0$, gives $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$. Corollary 3 is the case $r \equiv 1$. (Theorem 1 does not use the monotonicity of ϵ_n , only $\epsilon_\infty > 0$; for a schedule that may loosen one localizes on $\{\epsilon_n \geq 1/j \text{ for all large } n\}$ instead.) $\square$

Remark 1. Theorem 1 uses nothing about the form of $\tilde{q}_i$, and Corollary 2 adds only (9). Both are therefore statements about the algorithm as implemented, not about an idealized variant of it: any sampler whose draw mixture $\bar{q}_n^*$ is tracked by the denominator $\bar{q}_n$ in the ratio sense of (9) is covered by the corollary, the archive rule, the capped rejection and the underflow redraws included.

B.3 Deadline censoring

At the wall-clock deadline the run has proposed n' candidates, of which the set R still in flight, $|R| \leq W$, has no discrepancy; the estimate is computed from the $n = n' - |R|$ completed ones. Let $\hat{\pi}_{n'}(f)$ be the estimate the launched set would give at the same bandwidth and denominator, i.e. with the sums in (6) extended over all n' candidates, the unobserved discrepancies included.

Proposition 9 (deadline censoring). *Under Assumptions 1–3, for every bounded f ,*

$$|\hat{\pi}_n(f) - \hat{\pi}_{n'}(f)| \leq \frac{2\|f\|_\infty |R|/\delta}{\sum_{i \leq n'} W_{i,n}} \leq \frac{2\|f\|_\infty W}{\delta n \bar{W}_n}, \quad \bar{W}_n := \frac{1}{n} \sum_{i \leq n} W_{i,n}.$$

The censoring is therefore negligible whenever $W/n \rightarrow 0$ and the average weight stays bounded below, and it is $o(n^{-1/2})$ whenever $W = o(\sqrt{n})$.

Proof. Write W_i for $W_{i,n}$, $A = \sum_{i \leq n', i \notin R} W_i f(\theta_i)$, $B = \sum_{i \leq n', i \notin R} W_i$, $a = \sum_{i \in R} W_i f(\theta_i)$ and $b = \sum_{i \in R} W_i$. Then $\hat{\pi}_{n'}(f) - \hat{\pi}_n(f) = (A+a)/(B+b) - A/B = (a - (A/B)b)/(B+b)$, and $|a - (A/B)b| \leq \sum_{i \in R} W_i |f(\theta_i) - A/B| \leq 2\|f\|_\infty b$. Lemma 6 gives $b \leq |R|/\delta \leq W/\delta$, and $B+b \geq B = n\bar{W}_n$. $\square$

The launched set is the one the filtration of Lemma 5 indexes, so Theorem 1 and its corollaries apply to $\hat{\pi}_{n'}$, and the proposition transfers them to the completed-set estimate the implementation reports. The length bias of deadline-limited sampling (§8) thus acts on the estimate in two ways. The bound controls the direct effect through the at most W censored candidates; the indirect effect acts through the proposal sequence, whose archive is formed from arrived particles and so leans toward regions that simulate quickly. This second effect is part of $\tilde{q}_i$ and is corrected by the importance weighting up to r . On the five Cellular Potts production replicates, $W = 48$ and $n \approx 12,900$, so $W/n \approx 0.4\%$; the worst-case factor $1/\delta \approx 42$ makes the displayed bound loose at that budget, and the drain-after-deadline experiment of Appendix F, which lets the in-flight simulations finish, measures the effect on the posterior mean at 0.013 or less.

Remark 2 (growing snapshot count). Nothing above requires m to be fixed except the compactness and fixed dimension D of P_q , and the fixed floor δ . The implementation ties δ to m as $0.5/(m+1)$, so the two must be decoupled before m is allowed to grow, since Lemma 6's bound is $1/\delta$. With δ fixed and $m = m_n \rightarrow \infty$, run the proof of Lemma 7 with an n -dependent net: the union bound costs $D_n \log(1/\eta_n)$ with $D_n = \Theta(m_n)$, and the Lipschitz constant over the mixture weights grows at most polynomially in m_n , so choosing $\eta_n \rightarrow 0$ fast enough leaves the Borel–Cantelli step summable for any $m_n = o(n/\log n)$. In that regime factor (b) of r (§A.1) vanishes whenever $V_n/m_n \rightarrow 0$, with $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ the total variation of the proposal path: the draw-proportional m_n -point mixture is then a converging quadrature of the empirical mixture. The retroactive pass costs $\mathcal{O}(nm_n k)$.

B.4 Proof of Proposition 4

Write π for π_{ϵ_∞} . For bounded f , since $\pi^r(f) = \pi(fr)/\pi(r)$ and $\pi(r - \pi(r)) = 0$,

$$\pi^r(f) - \pi(f) = \frac{\pi(f(r - \pi(r)))}{\pi(r)} = \frac{\pi((f - c)(r - \pi(r)))}{\pi(r)} \quad \text{for every constant } c.$$

The supremum of the modulus over $\|f\|_\infty \leq 1$ is attained at $f = \text{sign}(r - \pi(r))$ and equals $\pi(|r - \pi(r)|)/\pi(r)$; halving gives the identity. For the bound, the first form also reads $[\pi(f(r-1)) - \pi(f)\pi(r-1)]/\pi(r)$, whose modulus is at most $2\|f\|_\infty\zeta_0/(1-\zeta_0)$ because $\pi(r) \geq 1-\zeta_0$; the supremum over $\|f\|_\infty \leq 1$, halved, is $\zeta_0/(1-\zeta_0)$. Under the set hypothesis $\zeta_0 \leq \eta_A\pi(A) + M\pi(A^c) \leq \zeta$, and $x \mapsto x/(1-x)$ increases on $[0, 1)$. For $\zeta_0 \geq 1$ only the trivial $\|\cdot\|_{\text{TV}} \leq 1$ survives. $\square$

Lemma 10 (prior-floor factor). *With $u = q_\infty/\pi$ the limiting proposal-to-prior ratio and $\nu_\infty < \delta$ the limiting bootstrap share, the prior floor’s contribution to r in §A.1(a) is*

$$r_{\text{floor}} - 1 = \frac{\nu_\infty + (1 - \nu_\infty)u}{\delta + (1 - \delta)u} - 1 = \frac{(\delta - \nu_\infty)(u - 1)}{\delta + (1 - \delta)u},$$

and on $\{u \geq c\}$, splitting at $u = 1$ and bounding each branch, $|r_{\text{floor}} - 1| \leq (\delta - \nu_\infty) \max\{(1 - \delta)^{-1}, (\delta + (1 - \delta)c)^{-1}\}$, which is the η_A of Proposition 4 for $A = \{u \geq c\}$. The factor is identically 1 when $\nu_\infty \geq \delta$, and $\rightarrow 1$ as $\delta \rightarrow 0$.

Proof. Evaluate (9) for the floor alone: the draw mixture is $\nu_\infty\pi + (1 - \nu_\infty)q_\infty$ and the denominator $\delta\pi + (1 - \delta)q_\infty$; divide both by π and subtract one. $\square$

B.5 Status of each assumption in the implementation

C Implementation Details

Relation to classical ABC-SMC. Table 6 sets the method against the generation-staged ABC-SMC/PMC implementation we compare with (the pyABC baseline), property by property. None of the listed properties is intrinsic to ABC-SMC as a family (smooth kernels and richer proposals exist), but each characterizes pyABC as configured. Because all state is a pure function of the history (last row), there is nothing to synchronize between workers, which fits the method to the asynchronous island model of §5.2.

Carried state and restart. Between calls the running sampler carries only the kept snapshots of §3.5 and the scheduler’s two throttles, the search cadence and the acceptance gate described below. A restart therefore reproduces the estimator over whatever history was evaluated, not the trajectory the run would have taken (kill-and-resume, Appendix D).

Hyperparameters. Table 7 lists every setting of the sampler with its option name in the implementation and the value used in the experiments.

Glyphs of Fig. 1. The toy problem has $k = N = 12$. Row (a) is textbook PMC with a quantile schedule and a hard threshold, weights $\omega_i^{(t+1)} \propto \pi(\theta_i)/q_t(\theta_i)$ normalized over the N kept. Point sizes vary where weights do (a1–a3 by $\omega^{(t)}$, a6 by $\omega^{(t+1)}$, b3 by the archive weights $\bar{W}_j$); shading in b1 is the discrepancy (light: far); dotted ellipses are Σ_t and Σ_n ; gray contours are the current proposal in a5 and b5 and the stored snapshots in b4 and b6; the dashed ellipse in row (a) is the hard acceptance region; the inset in column 2 is the kernel over ρ , the Gaussian K_{ϵ_n} in (b) against the indicator in (a). The thin box in every panel is the prior box, identical throughout. Bootstrap draws are prior draws with $w^* = 1$ and no stored bandwidth.

Bandwidth schedule. The schedule is a function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum ϵ_{hist} (§3.3), so the effective bandwidth can only decrease. Let μ denote the extra-individual margin (default $\mu = k$); it is unrelated to the snapshot count m of §4. The bandwidth is held until $k + \mu$ evaluated particles lie within it (the acceptance gate). After that, a search proposes the largest ϵ at which the kernel-weighted effective sample size of those particles retains a target fraction (default 0.95) of its current value, tightening by at most `max_tighten_factor` per search. The search runs on a rank’s first call, then once per `bisect_interval` ($= k$) of its own calls, and again whenever the rank’s cached history is rebuilt after an out-of-order arrival; asynchronous arrivals cause such rebuilds and lockstepped ones do not, which is why the barrierized twin’s bandwidth tightens less often than the asynchronous sampler’s at equal call counts (§6.1).

Table 5: What the implementation guarantees, what it assumes, and what it violates. “By construction” means the code enforces the property for every run, not that it was checked post hoc.

Assumption	Status	Basis
1 kernel, Lipschitz part	by construction	Gaussian and Epanechnikov are peak-normalized and Lipschitz in ϵ on every $[a, \epsilon_0]$, with $L_K(a) = 2/ea$ and $2/a$. The hard indicator is not, and is excluded from the theory.
1 $Z_{\epsilon_\infty} > 0$	holds for Gaussian	The Gaussian kernel is strictly positive, so $L_{\epsilon_\infty} > 0$ and the limiting target is non-degenerate whatever the discrepancy distribution. For a compactly supported kernel (Epanechnikov) it requires the achievable discrepancies to reach inside the support, which we do not verify.
2 denominator class	by construction	The prior support is a box and the archive lies in it, so component means do; the covariance carries an additive jitter $10^{-9}\text{tr}(\hat{\Sigma})/d + 10^{-12}\bar{b}^2$ ($\bar{b}^2$ the mean squared box width), whose second term is a run-independent floor, giving $\sigma_-^2 > 0$, and σ_+^2 follows from the archive lying in a bounded box. The truncation <i>normalizer</i> is the product of per-axis in-box masses, so with a correlated covariance $\bar{q}_n$ does not integrate to one. Assumption 2 does not require that: it constrains a positive function, and the discrepancy is carried by r instead (§A.1(c)).
4 execution model	holds; implementation property	(B1), (B2): the propagator draws from its own stream, seeded once per rank, and every simulator call receives a fresh seed derived from the run seed, the rank and a per-rank call counter, so the two streams are independent in the sense in which pseudo-random streams are, and no two calls share a seed. (B3): a worker proposes when its own simulation completes and builds the proposal from the results that have reached it; delivery is asynchronous message passing, exogenous to both streams. Lemma 5 then gives the adapted filtration with $\bar{q}_i$ the emitted density. What asynchrony leaves in r is the arrival-order reconstruction of the snapshots, a boundary of at most W particles per snapshot (§A.1(d)).
3 prior floor	by construction	The retroactive denominator floors the prior component’s mass at $\delta = 0.5/(m+1)$ (Appendix C), so $\bar{q}_n \geq \delta\pi$ and Lemma 6 bounds every posterior weight by $1/\delta$, independently of what the sampler does.
5 fidelity (9)	assumed; two of its four factors measured	The one place the sampler enters for Corollary 2; Theorem 1 does not use it. Decomposed into four factors in §A.1, two of them estimated from stored histories rather than bounded; since the factors multiply, the measured part is a partial accounting rather than a bound on the whole.
$\{\epsilon_\infty > 0\}$ (the event of the limit statements)	floor binds on g-and-k and Cellular Potts; never met on Gaussian mean and Lotka–Volterra	The event concerns the reporting bandwidth only; the sampler’s own bandwidths enter through $\bar{q}_i$, on which no condition is placed. A strictly positive <code>min_tol</code> forces $\epsilon_\infty \geq \text{min_tol} > 0$, since $\{\epsilon_n\}$ is non-increasing and clipped from below, which is all Theorem 1 and Corollary 2 need. The floor acts only through that clip, so a run whose final bandwidth lies above it is path-identical to the floored run. Final bandwidths over ϵ_0 : g-and-k $2.2\text{--}3.0 \times 10^{-3}$, Cellular Potts production $1.3\text{--}3.2 \times 10^{-3}$ (and 1.2×10^{-3} or more at $\epsilon_0 = 10$; the 80^3 runs never left ϵ_0), g-and-k calibration trials $1.1 \times 10^{-2}\text{--}0.55$ (a replayed sample of 30 of the 1,000, on 6 workers rather than 48), all above a floor of $10^{-4}\epsilon_0$; Gaussian mean $6\text{--}7 \times 10^{-6}$ and Lotka–Volterra $1.3\text{--}2.4 \times 10^{-7}$, below it. The bandwidth a finite budget reaches is not an asymptotic ϵ_∞ : no run reached its floor, and the effective-sample-size ceiling (§4) is the consequence.

The implementation also offers quantile, geometric-decay and acceptance-rate rules (`scheduler_type`); they act only under the hard kernel, and with a smooth kernel each reduces to the rule above, so the option has no effect in any reported run.

Proposal covariance. The archive A_n is the top- k history members by smallest discrepancy. Component weights are formed in log-space as $\tilde{W}_j \propto w_j K_{\epsilon_n}(\rho_j)$ and normalized (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is $\Sigma_n = s[\widehat{\text{Cov}}_{\tilde{W}}(A_n) + \lambda_n I]$ with s the `perturbation_scale` and a two-part jitter $\lambda_n = 10^{-9} \text{tr} \widehat{\text{Cov}}_{\tilde{W}}(A_n)/d + 10^{-12} \bar{b}^2$, where $\bar{b}^2$ is the mean squared prior-box width; it is Cholesky-factored once per call. The first term is scale-aware, so a tightly converged archive (marginal standard deviation $\sim 10^{-4}$) is not swamped by the floor as a fixed 10^{-6} would swamp it; the second is a run-independent absolute floor that keeps the matrix factorable for a degenerate archive and supplies the uniform lower spectral bound Assumption 2 needs (Table 5). Each mixture component is truncated to the prior box, with its log normalizing mass computed as the product of the per-axis in-box masses. That is exact for a diagonal covariance, in which case the proposal integrates to one on the support; for a correlated covariance it is an approximation, so the evaluated proposal is a positive function

Table 6: A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize pyABC as configured, not ABC-SMC as a family. The running sampler does carry the online snapshot buffer of §3.5; the posterior estimate rebuilds it from the history.

Property	Classical ABC-SMC	This work
Update style	generation, barrier-synchronized	event-driven (single-arrival), no barrier
Archive	explicit, generational	reconstructed, sliding top- k
ABC likelihood	hard threshold	smooth kernel
Importance weight	against q_{t-1} only	balance heuristic over $\mathcal{S}$
Archive state	mutable, carried	none; reconstructed per call
Posterior estimate	final population	pure function of the history

Table 7: Settings of the asynchronous sampler. Option names are those of the implementation; values are those of the production experiments, except where a study sweeps them (Appendix F).

Symbol	Meaning	Option	Value
k	archive size	<code>k</code>	100 (200 in the four-dimensional calibration runs)
ϵ_0	starting bandwidth	<code>tol_init</code>	5 (Gaussian mean), 10 (g-and-k), 5×10^5 (Lotka-Volterra), 0.1 (Cellular Potts; 10 in its earlier parameterization)
K_ϵ	ABC kernel	<code>kernel</code>	Gaussian
s	scale of the perturbation covariance Σ_n	<code>perturbation_scale</code>	0.8
S	kept snapshots	<code>amis_snapshots</code>	20
	snapshot cadence, in calls	<code>amis_interval</code>	k
	ESS retention per search	<code>ess_target</code>	0.95
	largest tightening per search	<code>max_tighten_factor</code>	0.5
	search cadence, in calls per rank	<code>bisect_interval</code>	k
μ	acceptance gate: hold until $k + \mu$	<code>additional_needed_inds</code>	k
	particles lie within ϵ		
	bandwidth floor; a positive value puts the run on $\{\epsilon_\infty > 0\}$ (§4)	<code>min_tol</code>	$10^{-4}\epsilon_0$ recommended; unset in the reported runs, where that value never binds on g-and-k or Cellular Potts (§4)
δ	prior floor in $\bar{q}_n$	—	$0.5/(m+1)$

that need not integrate to one. This is why §4 states its denominator assumption for positive functions and assigns the discrepancy to factor (c) of r (§A.1).

Matched pyABC acceptor. For pyABC, pyABC’s `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy ρ with probability $K_\epsilon(\rho)$ (peak-normalized so $K_\epsilon(0) = 1$), using the *same* kernel K_ϵ as the asynchronous sampler; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5.2).

Bounded-memory posterior estimator. The retroactive AMIS estimator (6) evaluates the snapshot denominator $\bar{q}_n$, a draw-proportional mixture of $m \leq S+1$ history-reconstructed proposals (21 at the $S = 20$ of every experiment) with a defensive prior component (mass floored at $\delta = 0.5/(m+1)$), at each of the n evaluated particles. The total cost is $\mathcal{O}(nmk) = \mathcal{O}(nk)$ with m fixed. The snapshot indices are spread over the n_{arch} archive-phase calls (indices 0, step, 2 step, $\dots$ with $\text{step} = \max(1, \lfloor n_{\text{arch}}/S \rfloor)$), plus the most recent archive-phase proposal if it is not already among them; hence $m \leq S+1$ rather than S , and $m = 2$ when $S = 0$. Each is rebuilt from the history prefix the call saw, so the pass never reads the online ring buffer of §3.5. For near-instantaneous simulators the history reaches $\mathcal{O}(10^7)$ particles, so the pass is accumulated in chunks of $2^{16} = 65,536$ history points: peak memory is $\mathcal{O}(\text{chunk} \cdot m \cdot k)$ rather than $\mathcal{O}(n)$. Each particle’s denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across*

chunk boundaries), so the result is bit-identical to the unchunked computation. Benchmarked at $k = 100$, $m = 20$ over n from 2.5×10^4 to 10^6 , the pass costs 49.3 s per 10^6 particles on a single core and is linear in n , so the 3×10^7 -particle Gaussian history extrapolates to ≈ 25 minutes; peak memory is flat in n , as the chunking predicts. The pass runs after the timed budget and is excluded from the throughput measurements; for a near-instantaneous simulator it is comparable to the timed budget itself, for cost-bearing simulators negligible against the simulation time.

D Additional Experimental Protocol

Effective simulating ranks. A “ W -worker” point allocates W physical cores to both samplers. Every rank of the asynchronous sampler evaluates; pyABC’s MPI sampler is root-driven, so rank 0 dispatches without simulating and pyABC has $W-1$ simulating ranks. We report the allocated W for both, which flatters the asynchronous sampler by $1/W$ (2.1% at $W=48$, 0.3% at 384) and is not corrected.

Seeding. Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`-extend`) runs.

Crash recovery (kill-and-resume). On the Gaussian-mean benchmark (4 workers, a 1.2×10^5 -simulation budget) a run hard-killed partway through inference and relaunched with an identical command reloads the periodic population checkpoint, completes the same total simulation budget with no duplicated records, and recovers the analytic posterior mean (absolute error 7.6×10^{-3} , against 7.5×10^{-4} for an uninterrupted run). It is not a bit-identical replay (the two posterior means differ by 6.9×10^{-3}), because the candidate generator is re-seeded rather than checkpointed, MPI arrival order is not deterministic, and the online AMIS snapshot buffer is rebuilt empty on restart. The post-restart proposal sequence therefore follows a different, equally valid adaptation trajectory. This is the distinction §3.1 draws: recovery yields a valid posterior over the actual evaluated history, not the identical one.

Cluster and wall-clock control. Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All wall-clock-budgeted methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serializes its current state into the records and returns, so a partial final generation is recorded rather than discarded (the simulation-limited studies marked † in Table 8, the SBC and kill-and-resume runs, instead stop at a fixed per-trial simulation count). Posterior estimation and plotting run off the timed inference path, so they do not count against the budget. Matched-budget tolerances (§5.3) count only records of completed simulations; pyABC also writes one record per accepted particle, and mixing the two would flatter it.

Benchmark configurations. Table 8 lists the resolved settings (k is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size $k \in \{100, 1000\}$; all other studies fix the workers shown.

Benchmark-specific notes. Each benchmark fixes one observed dataset (generated from seed 42) and infers parameters under uniform priors; the discrepancy is a distance between simulated and observed summary statistics. *Gaussian mean*: $n_{\text{obs}} = 100$ observations $\sim \mathcal{N}(0, 1)$, prior $\mu \sim \mathcal{U}[-5, 5]$, summary the sample mean, discrepancy $|\bar{x}_{\text{sim}} - \bar{x}_{\text{obs}}|$; the closed-form Gaussian posterior is the calibration and recovery reference. *g-and-k*: the quantile-defined g-and-k distribution ($c = 0.8$), $n_{\text{obs}} = 1000$, true $(A, B, g, k) = (3, 1, 2, 0.5)$, priors $A \sim \mathcal{U}[0, 6]$, $B \sim \mathcal{U}[0.1, 4]$, $g \sim \mathcal{U}[0, 5]$, $k \sim \mathcal{U}[0, 1]$; summary the seven octiles (the 0.125, ..., 0.875 quantiles), discrepancy Euclidean. *Lotka–Volterra*: a Gillespie stochastic simulation (Gillespie, 1977) of the four-reaction predator–prey system from $(X_0, Y_0) = (50, 100)$ over $T = 30$, true rates $\theta = (0.5, 0.025, 0.025, 0.5)$, priors $\theta_1, \theta_4 \sim \mathcal{U}[0.05, 2]$ and $\theta_2, \theta_3 \sim \mathcal{U}[0.001, 0.2]$; summary the six

Table 8: Resolved experiment configurations. Workers, per-run wall-clock budget, replicates (trials for SBC), archive size k , and methods (A: asynchronous; S: pyABC (matched kernel); R: rejection ABC; T: barrierized twin). The twin row covers the straggler, heterogeneity and Cellular Potts 50^3 studies at 16, 48 and 48–384 workers. [†] Simulation-limited: the budget is per-trial (SBC) or per-run (kill-and-resume) *simulations*, not wall-clock seconds; the twin, and the asynchronous re-run matched to it, stop at a fixed simulation count per level (§5.2, Table 9). [‡] Also run with AMIS on/off, a drain-after-deadline variant, and a steeper coupling regime (base delay 0.5 s, cap 8 s). [§] The Cellular Potts rows of these studies ran the benchmark’s earlier configuration (Appendix E).

Experiment	Workers	Budget (s)	Reps	k	Methods
Gaussian mean	48	300	5	100	A,S,R
g-and-k	48	600	5	100	A,S,R
Lotka–Volterra	48	600	5	100	A,S
Cellular Potts 50^3	48	3600	5	100	A,S,R
Cellular Potts 80^3	48	3600	3	100	A,S,R
Straggler	16	300	5	100	A,S
Runtime heterogeneity	48	60	5	100	A,S
Parameter-coupled runtime [‡]	48	60	5	100	A,S
Strong scaling (LV)	1–288	180	5	100/1000	A,S
Strong scaling (CPM) [§]	1–384	1800	3–5	100	A,S
Barrierized twin [§]	16 / 48 / 48–384	sims [†]	5	100	T,A
Sensitivity	48	300	5	50/200	A
Ablation	48	300	5	100	A
SBC (Gaussian, 1-D)	48	20k [†]	1000	100	A,S
SBC (bimodal)	48	20k [†]	1000	100	A,S
SBC (g-and-k, 4-D)	48	40k [†]	1000	200	A,S
SBC (g-and-k, 2×)	48	80k [†]	500	200	A,S
Kill-and-resume	4	120k [†]	1	100	A

statistics ($\bar{X}$, sd X , $\bar{Y}$, sd Y , $\log(1 + \text{var } X/\bar{X})$, $\log(1 + \text{var } Y/\bar{Y})$), discrepancy Euclidean. *Cellular Potts*: the real `cellsInSilico` (NASTJA) simulator; the setup and the screening that fixed it are in Appendix E. Its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

Computing environment. All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and ≈ 94 GB memory per node, connected by Mellanox EDR InfiniBand. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT (Flamary et al., 2021) for the sliced Wasserstein distance; the asynchronous sampler runs as a propagator in a fork of Propulate. Unless noted, ranks are placed one per physical core, so a “ W -worker” point occupies W physical cores packed onto $\lceil W/48 \rceil$ fully-reserved nodes. The production experiment of the three cheap benchmarks (Gaussian mean, g-and-k, Lotka–Volterra) instead placed its 48 ranks as 12 per node on four nodes under whole-node reservation, after the single-node submissions failed, which is why its asynchronous throughput is below the packed single-node scaling run’s on the same benchmark (§6.3). The Cellular Potts production run and the straggler, heterogeneity, twin and scaling experiments are single-node or fully packed. Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

E The Cellular Potts inference setup

The benchmark runs the `cellsInSilico` Cellular Potts tissue model (Berghoff et al., 2020) through the NASTJA framework (Berghoff et al., 2018): a 50^3 lattice for 501 steps (an 80^3 lattice for 1001 steps in the larger configuration), seeded with a small cluster of cells that divide and move, summarized at the final step.

The inference problem was designed on a screening experiment of about 12,000 simulations over space-filling designs of six candidate parameters, scored by the ratio of between-parameter to within-parameter variation of each summary block. Of the seven summary blocks the shipped configuration weighted equally, two scalars, the log cell count and the log 95th-percentile radius, carry nearly all of the signal (signal-to-noise 12 and 9 against about 1 for the four ten-dimensional curve blocks), so the discrepancy is taken over those two, equally weighted. One evaluation is the feature-average of four replicate simulations at the same parameters, because a single realization is too noisy for the discrepancy to order nearby parameters. The two summaries resolve exactly two directions (how many cells there are, and how large the cluster is at a given count), so the inferred parameters are the two that move them independently: the division rate (count) and the target cell volume (radius at fixed count), whose responses are near-orthogonal ($|\cos| = 0.07$). Every other candidate expresses itself through the count: cell motility, adhesion and surface tension are confounded with the division rate at $|\cos| \geq 0.8$ at 50^3 , and at 80^3 , with $6.4\times$ the cells, the confounding sharpens rather than breaks ($0.81 \rightarrow 0.98$ for the surface term). The benchmark is therefore two-dimensional as a property of the model, not for want of screening. Priors are log-uniform on $[0.001, 0.2]$ and $[200, 1200]$ with truths 0.009 and 500; motility is fixed at 1400. Rejection ABC on 2000 prior draws forecast 91% and 64% contraction for the two parameters before any adaptive run, against the 92% and 81% the asynchronous estimator reports (§6.4).

Two configurations. The *production* configuration is the two-parameter setup above and supplies every posterior result, Tables 2 and 4, the 80^3 systems comparison, and the theory diagnostics of Appendix A. The *earlier* configuration was used by the 50^3 twin and scaling experiments (the 50^3 points of Fig. 2c, Table 11, Fig. 3 right, Fig. 13). It inferred the cell division rate and motility on normalized $[0, 1]$ priors mapping to $[6\times 10^{-5}, 0.6]$ and $[0, 10^4]$ (true normalized values 0.0499 and 0.2), scored one simulation per evaluation (5.1 s at 50^3 , in-run coefficient of variation 0.10) by a whitened distance over morphological summary features (radial density and pair-correlation profiles, free-area and structure-factor profiles, a DBSCAN dispersion fraction, and log cell count and radius). The screening above found those two parameters confounded, which is why the production configuration replaced them. The systems measurements depend on the configuration only through its runtime distribution, which the predictor takes from the run itself.

F Additional results

The studies below support the main text and are reported in full here.

F.1 Runtime heterogeneity: archive quality

The synthetic instance. The heterogeneity study shows the same conversion where the runtime law is known (Fig. 7): at a fixed 60 s budget on the Gaussian benchmark pyABC completes fewer simulations as σ grows and its archive’s posterior-mean error to the analytic posterior climbs from ≈ 0.05 to ≈ 0.28 , while the asynchronous sampler completes several-fold more and its archive holds ≈ 0.007 – 0.011 . These are archive means; the posterior estimates of both samplers in this experiment are scored in Table 10.

F.2 The barrierized twin on the straggler benchmark, and pyABC under injected heterogeneity

A persistent straggler. One of 16 workers is given a permanent post-evaluation delay of 0.1 s scaled by 0, 1, 5, 10, 20 (Table 9). The generation time is the slow worker’s recorded evaluation time (2.000 s at $20\times$) plus the 3.6 ms per-evaluation overhead measured on the fast workers. The twin therefore cannot exceed $16/2.004 = 7.98$ simulations per second. Dividing the measured asynchronous rate by that ceiling gives $401\times$, against a measured $402\times$. The agreement is the same at $5\times$ and $10\times$. At $0\times$ and $1\times$ the prediction fails: with no straggler to wait for, the collective itself costs about 0.3 s per barrier on a 4 ms workload, a term the model does not contain, and coarsening the barrier to one per 112 evaluations (pyABC’s population) amortizes it. From $5\times$ up the two barrier granularities agree with each other and with the prediction within 0.5% (0.2% at $10\times$, 0.06% at $20\times$): once a straggler binds, the barrier’s cost does not depend on how often it is taken.

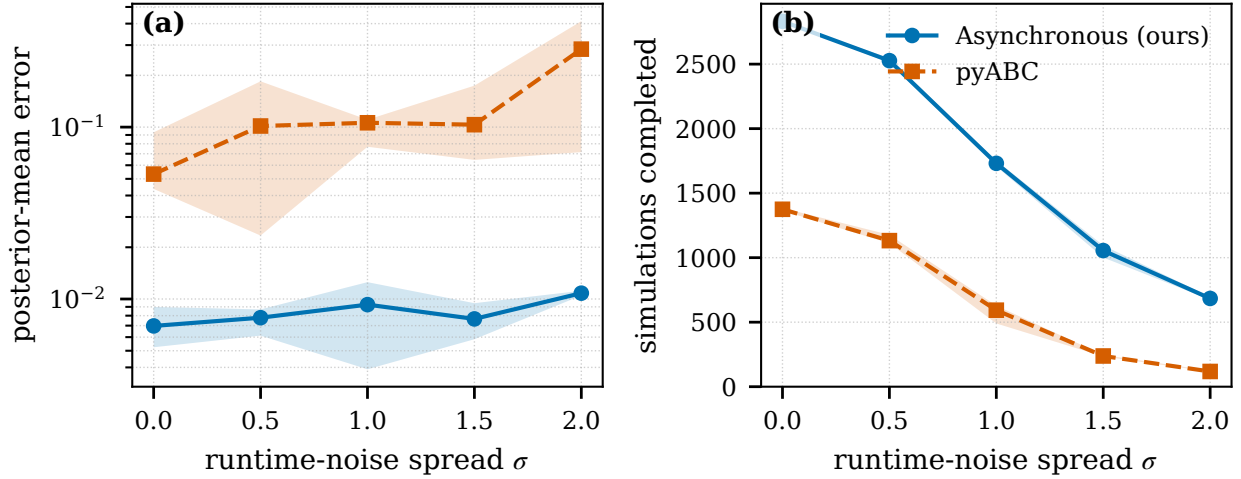


Figure 7: Inference per unit wall clock under runtime heterogeneity (Gaussian mean, analytic posterior, 48 workers, 60s budget, five replicates; bands are inter-quartile ranges). Left: error of the archive mean (the unweighted top- k archive, not $\hat{\pi}_n$; §6.4) to the analytic posterior mean, against the runtime spread σ . Right: simulations completed within the budget. The barrier starves pyABC as σ grows; the asynchronous sampler’s error stays flat.

Table 9: Barrierized twin on the straggler benchmark ($W = 16$, five replicates, medians). Throughput in simulations per second; *predicted* as in §5.3, and at $0\times$ and $1\times$ the collective’s latency, absent from the model, dominates. Lower block: W_1 of each sampler’s posterior to the analytic posterior at the twin’s simulation count and seeds, at the bandwidth its schedule reached and, last row, re-reported at $\epsilon_{(100)}$ (§6.4).

Straggler slowdown	$0\times$	$1\times$	$5\times$	$10\times$	$20\times$
Asynchronous, sims/s	3888	3112	3241	3218	3205
Twin, barrier every W	48.7	65.3	31.6	15.9	8.0
Twin, barrier every 112	162.8	148.4	31.8	15.9	8.0
Twin, predicted	(3870)	(154)	31.8	15.9	7.98
Ratio, measured (every W)	$80\times$	$48\times$	$103\times$	$202\times$	$402\times$
Ratio, predicted	—	—	$102\times$	$202\times$	$401\times$
<i>Posterior recovery, W_1 to the analytic posterior, at the twin’s simulation count</i>					
Evaluations (= calls per rank $\times 16$)	75,104	44,512	12,304	6,304	3,200
Bandwidth reached, asynchronous	5×10^{-4}	7×10^{-4}	0.003	0.006	0.16
Bandwidth reached, twin every W	5×10^{-4}	8×10^{-4}	0.06	1.3	2.9
$\epsilon_{(100)}$ of the same history	3×10^{-4}	5×10^{-4}	0.002	0.003	0.008
Asynchronous at that count	0.011	0.009	0.010	0.010	0.068
Twin, barrier every W	0.013	0.007	0.016	0.789	1.713
Twin, barrier every 112	0.009	0.009	0.009	0.017	1.082
Twin every W , re-reported at $\epsilon_{(100)}$	0.015	0.010	0.010	0.010	0.010

The schedule must move. The twin shows the bandwidth transient of §6.4 on a cheap simulator. At the twin’s own simulation count and seeds the asynchronous sampler reports $W_1 = 0.010$ and 0.068 to the analytic posterior at $10\times$ and $20\times$ where the twin reports 0.79 and 1.71 (Table 9, lower block). The propagator is the same, and so are the 200 calls per rank at $20\times$. But the twin’s schedule reached a bandwidth of 2.9, against 0.16, on histories whose 100th-smallest discrepancy is 0.008, because under lockstep it takes fewer search steps (Appendix C). Re-reporting the twin’s histories at $\epsilon_{(100)}$ brings them to 0.010–0.015.

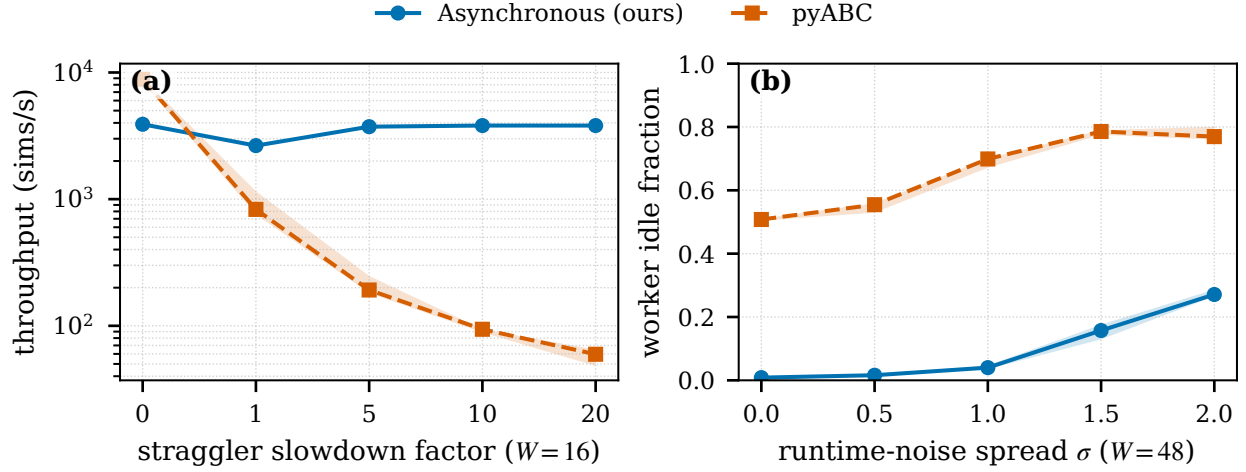


Figure 8: pyABC against the asynchronous sampler under injected runtime heterogeneity on the Gaussian mean (§6.1). (a) Throughput under a persistent straggler: one of 16 workers carries a permanent post-evaluation delay of 0.1 s scaled by the factor (300 s; medians and inter-quartile ranges over five replicates; logarithmic axis). (b) Fraction of worker time spent idle under a lognormal runtime multiplier of spread σ (48 workers, 60 s).

F.3 The barrierized twin under runtime heterogeneity and on Cellular Potts

Table 10 gives the twin’s heterogeneity experiment behind §6.1, with the posteriors of every sampler at the bandwidth its schedule reached and re-reported at the order statistic (§6.4); Table 11 factorizes the twin’s Cellular Potts slowdown into the barrier’s share and the unattributed duration ratio.

F.4 Tolerance curves

Figure 9 shows the curves Table 2 is read from.

F.5 Parameter-coupled runtime

The heterogeneity study above slows simulations independently of their parameters; a sharper test couples runtime to the *inferred parameter*, probing the completion-time selection of §8. On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) the post-evaluation delay is $d(\mu) = \min(d_0 e^{c(\mu-\mu_c)/\ell}, d_{\max})$ with $d_0 = 0.02$ s, $\mu_c = 0$, $\ell = 0.15$ and $d_{\max} = 0.5$ s, swept over $c \in \{0, 0.5, 1, 2\}$ ($c = 0$ is uniform runtime), so one region of parameter space evaluates far faster than its mirror image and the every-particle rule over-represents it in the raw archive. The coupling has the intended effect on the sampling rate: as it steepens, aggregate throughput falls by $\approx 43\%$ for the asynchronous sampler and more steeply for pyABC (Fig. 10a).

The posterior does not follow. The asynchronous posterior-mean error to the analytic posterior stays ≈ 0.01 at every coupling, comparable to pyABC, which discards latecomers (Fig. 10b). This holds with or without AMIS ($S=0$), for the unweighted archive and for the AMIS-reweighted estimator (6), and in a steeper regime (base delay 0.5 s, cap 8 s). The reweighting changes the spread, not the mean: it lowers the effective-sample-size fraction from ≈ 0.99 to ≈ 0.88 and widens the central 90% interval by $\approx 15\%$. A drain-after-deadline variant, which lets every in-flight simulation finish past the deadline, moves the weighted posterior mean by at most 0.013, within the replicate spread and with no systematic direction, so the deadline censoring is negligible at these couplings, as Proposition 9 predicts for $W/n \ll 1$. AMIS neither creates nor removes a mean bias here, and a correction in the manner of anytime Monte Carlo (Murray et al., 2021) is future work.

Table 10: Barrierized twin under runtime heterogeneity ($W = 48$, lognormal runtime multiplier of scale σ , five replicates, medians of simulations per second of each sampler’s own elapsed wall clock). *Predicted* is from the injected law and the asynchronous run’s measured overhead. The $\sigma = 0$ control costs almost nothing because these evaluations take 1 s and barrier latency is negligible against them; the cost is the wait for the slowest draw and grows with σ . pyABC degrades less than the twin because it discards latecomers instead of waiting for them. The lower block scores the posteriors against the analytic posterior at the bandwidth each schedule reached, then re-reported at the 100th order statistic of the same history (§6.4); no rank made k calls in the asynchronous run, so its schedule never ran at $\sigma \leq 0.5$, and the twin’s ran only at $\sigma \leq 0.5$.

Heterogeneity σ	0	0.5	1.0	1.5	2.0
Asynchronous, sims/s	47.6	42.1	28.9	17.6	11.4
Twin, barrier every W	44.8	15.1	4.37	1.06	0.37
Twin, barrier every 96	45.1	19.8	6.50	1.75	0.45
pyABC	23.1	18.9	9.91	4.35	2.39
Ratio, measured (every W)	$1.06\times$	$2.8\times$	$6.6\times$	$16.6\times$	$29.6\times$
Ratio, predicted	$1.00\times$	$2.77\times$	$6.34\times$	$14.0\times$	$35.4\times$
Ratio, pyABC	$2.1\times$	$2.2\times$	$2.9\times$	$4.1\times$	$4.8\times$
<i>Posterior recovery, W_1 to the analytic posterior (calls per rank: async 59–13, twin 440–10)</i>					
Evaluations, async	2832	2527	1731	1055	684
Evaluations, twin	21,120	6,912	2,112	672	480
Bandwidth reached, async	5	5	0.92	1.25	0.78
Bandwidth reached, twin every W	0.36	2.4	5	5	5
Async, at the schedule’s bandwidth	2.12	2.04	0.68	0.93	0.56
Twin every W , at the schedule’s bandwidth	0.19	1.43	2.20	2.22	2.28
pyABC, last generation	0.53	0.61	1.36	2.19	2.21
Async, re-reported at $\epsilon_{(100)}$	0.011	0.011	0.010	0.013	0.042
Twin every W , re-reported at $\epsilon_{(100)}$	0.015	0.013	0.012	0.040	0.203
Async archive (top 100)	0.018	0.017	0.016	0.014	0.019

Table 11: The twin’s slowdown on Cellular Potts 50³ (earlier parameterization, Appendix E), factorized (five replicates, medians). Throughput is utilization over mean simulation time, so the ratio is the product of the two columns that follow. The utilization ratio is what the barrier costs; it agrees with the straggler factor measured inside the twin’s own generations and with the prediction from the asynchronous run’s runtime distribution. The duration ratio (the twin’s simulations running longer, with replicate means from 6 to 36 s against 5.1 s on every asynchronous replicate) is not a barrier effect and is not attributed here.

Workers (= cores)	48	96	192	384
Throughput ratio, async / twin	$1.91\times$	$1.87\times$	$1.86\times$	$2.82\times$
= utilization ratio	$1.18\times$	$1.19\times$	$1.21\times$	$1.21\times$
× simulation-duration ratio	$1.63\times$	$1.59\times$	$1.52\times$	$2.33\times$
Straggler factor of the twin’s generations	1.21	1.28	1.21	1.29
Predicted from asynchronous timing	$1.26\times$	$1.30\times$	$1.32\times$	$(2.34\times)$

F.6 Strong scaling on a cheap simulator

Lotka–Volterra is an adversarial simulator for a scaling study. Each evaluation takes ≈ 4 ms, so there is little per-evaluation compute to distribute, and an instrumented run (Fig. 11) shows a worker inside the simulator $\approx 25\%$ of its time at the 48-worker throughput peak, falling to $\approx 8\%$ and $\approx 2\%$ as the job spreads from one node to three to six. The mechanism is the method’s single shared population: every completed evaluation is exchanged with all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. Aggregate throughput therefore rises almost linearly while the job fits on one node and then declines across fully packed multi-node jobs (Table 12), because adding ranks beyond one node lengthens the exchange faster than it adds useful work. pyABC is

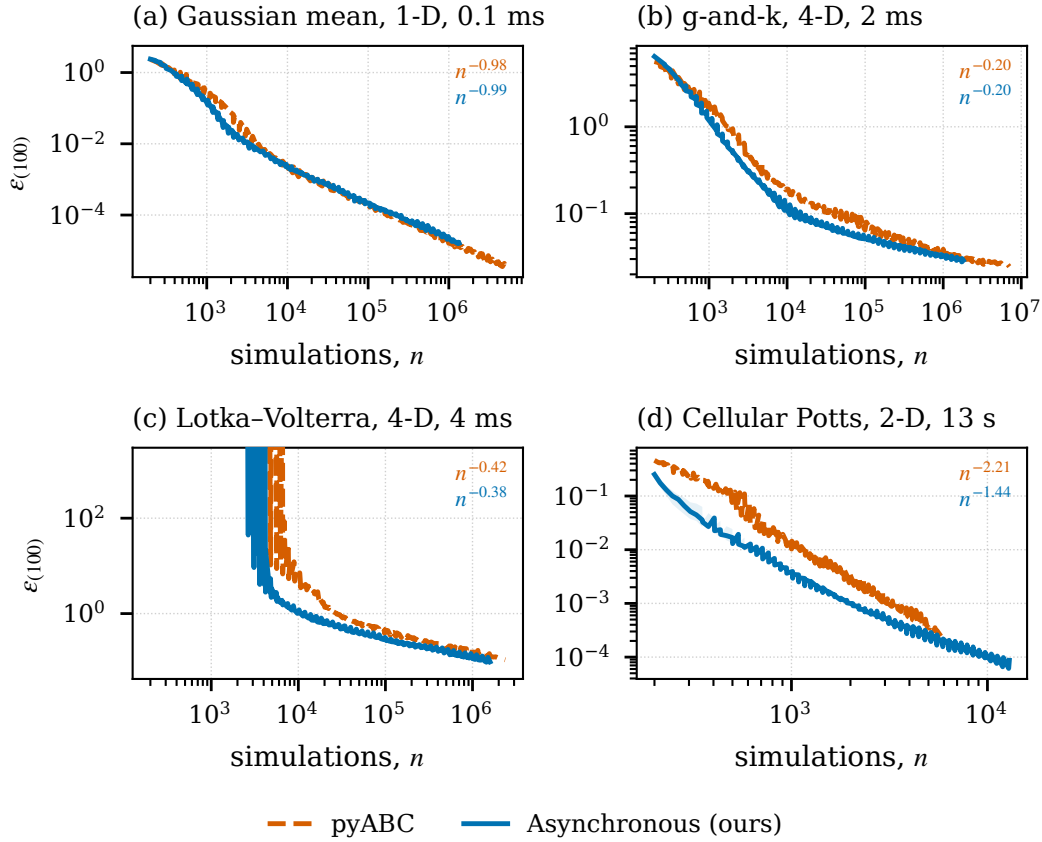


Figure 9: Tolerance against simulations spent, $\epsilon_{(100)}(n)$, asynchronous against pyABC on every benchmark (medians and replicate range over five replicates; the slope over the last decade of n in the legend). The vertical gap at equal n is the per-simulation efficiency, the horizontal extent is the extra simulations the wall clock allowed. On Lotka–Volterra 98% of simulations return the extinction fallback, so $\epsilon_{(100)}$ sits on that plateau until about 5000 draws have produced 100 survivors; the plateau is cut off.

given a population equal to the worker count at high core counts (a smaller population cannot occupy every core); it scales steadily and overtakes the asynchronous sampler beyond three nodes. This is the boundary of the asynchronous advantage on a cheap, homogeneous simulator, and it lifts once the simulator carries real per-evaluation cost (Fig. 3, right). The archive size moves the boundary, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$. Raising k to 1000 costs 39% of the single-node peak throughput. Sweeping k over $\{50, 100, 200, 400, 800, 1000\}$ resolves that cost into a smooth monotone decline, with a single step in the crossover: from beyond three nodes to three at $k \geq 400$ (Table 14).

F.7 Calibration against the archive size and the snapshot buffer

The calibration tests of §6.4 fix k and S per benchmark, which leaves open whether the two settings must be retuned as the dimension grows. To test this we sweep them directly on a d -dimensional Gaussian-mean target, $y_i \sim \mathcal{N}(\theta, \sigma^2 I_d)$ with $\theta \in \mathbb{R}^d$ under a flat box prior, whose posterior is known exactly at every d (SBC against that exact posterior returns 0.496/0.794/0.905/0.952, so the target itself is calibrated and any departure below is a property of the method). The grid is $d \in \{1, 2, 4, 8, 16\} \times k \in \{50, 100, 200, 400, 800\} \times S \in \{0, 5, 20, 50\}$, 1000 trials per cell, reporting the same full-history estimator (Table 13). S sets both the online buffer length and the number of reconstructed proposals in the posterior denominator (§3.5), and the sweep varies the two together; at $S = 0$ the denominator holds only the first and the last reconstructed proposal, so the $S = 0$ column tests the posterior-side mixture as much as the online buffer. At $S = 0$ the estimator

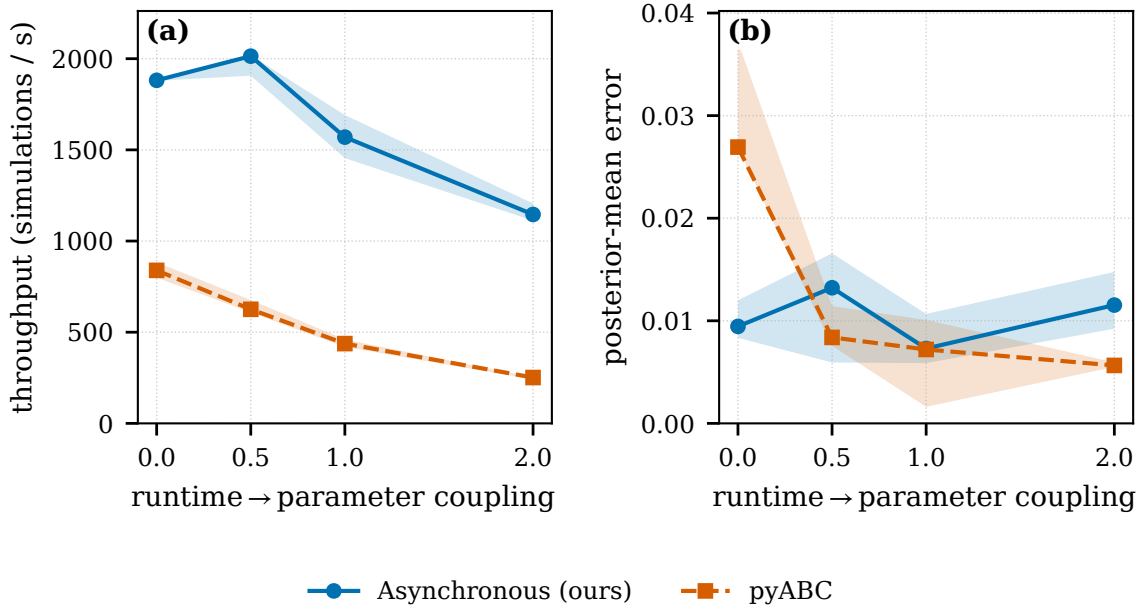


Figure 10: Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens (monotonically for pyABC, and from a peak at $c = 0.5$ for the asynchronous sampler), consistent with the fast region dominating the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous sampler and pyABC, which discards latecomers, so the over-representation does not measurably distort the posterior estimate at these coupling levels.

Table 12: Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 3, left, shows the inter-quartile spread). Asynchronous (ours) at both archive sizes vs. pyABC. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 pyABC is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous sampler peaks on one node and then declines as multi-node coordination dominates, so pyABC overtakes it beyond three nodes at $k = 100$. Enlarging the archive tenfold costs throughput at every scale, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$, and moves that crossover in from beyond three nodes to three.

Workers	1	4	16	48	144	192	240	288
Nodes	<1	<1	<1	1	3	4	5	6
Synchronous	105	96	437	2952	3757	4322	4539	6034
Asynchronous ($k = 100$)	149	369	1983	6240	4195	2615	2088	1729
Asynchronous ($k = 1000$)	106	133	620	3804	2804	1957	1587	1336

is over-confident in one and two dimensions (mean deviation -0.088); every $S \geq 5$ removes that deviation, and the experiments use $S = 20$.

The archive size does not, in this range, trade calibration against throughput. Because k has both a statistical effect (above) and a systems cost (per-arrival proposal reconstruction is $\mathcal{O}(k)$, so a larger archive is slower at every worker count; §6.3), one might expect to trade throughput for calibration. Over the range we can test, that expectation does not hold. Table 14 puts the throughput cost of k next to its worst-case miscalibration over the dimensions swept, using a throughput grid matched on everything

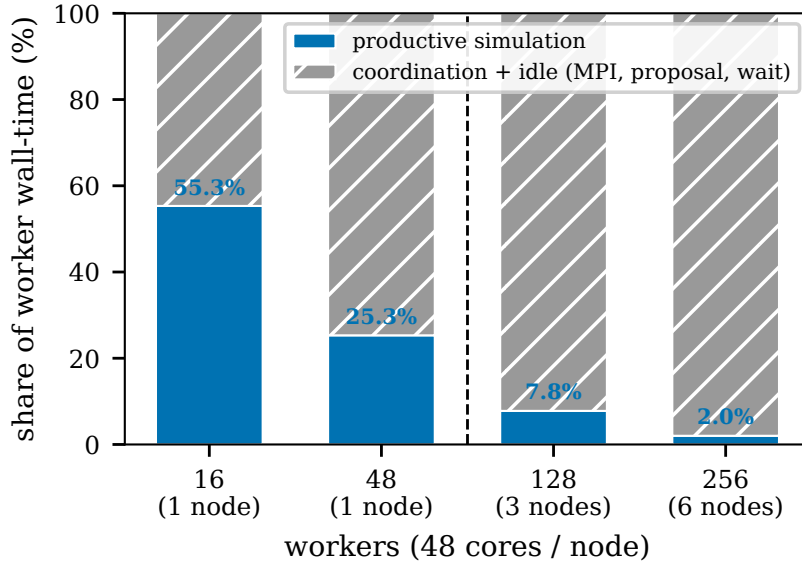


Figure 11: Measured decomposition of asynchronous worker wall-time on Lotka–Volterra ($k = 100$; 180 s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated), the share of worker wall-clock spent inside the simulator, collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 3 is therefore a measured property of coordinating a near-instantaneous simulator.

but k (Lotka–Volterra, 180 s wall cap, five replicates, worker counts 1–288: $k \in \{100, 1000\}$ from the main scaling run and $k \in \{50, 200, 400, 800\}$ from a dedicated k -frontier run sharing that configuration). Of the six archive sizes, only two are Pareto-optimal, and they are the two smallest: $k = 100$ is simultaneously the best-calibrated setting tested (worst deviation 0.064, at $d = 4$) and within 7% of the fastest (6240 vs 6684 simulations/s at the single-node peak). Every larger archive is dominated by it on *both* axes. At $k = 200$, the sampler is 5% slower and nearly three times worse calibrated (0.180). At $k = 400$, it is 14% slower with a deviation of 0.195. At $k = 800$, it costs 31% of throughput and remains slightly worse calibrated than $k = 100$ (0.074). The only faster archive, $k = 50$, is the one that fails calibration (0.250 at $d = 8$). The archive size therefore needs only to clear a threshold: above the small-archive failure region, further enlargement adds no calibration and costs throughput monotonically.

The two axes come from different benchmarks by design (calibration needs a target whose posterior is known at every d ; throughput needs the regime where an $\mathcal{O}(k)$ per-arrival cost is most exposed), so the throughput column is an upper bound on the relative cost of k and the conclusion is conservative. At scale k acts as a step in the crossover with pyABC at a population equal to the worker count: beyond three fully packed nodes for $k \leq 200$, three nodes for $k \geq 400$ (last column).

F.8 Worker utilization on Cellular Potts at 50^3

F.9 Lotka–Volterra extinction

The simulator has an absorbing state: if either population reaches zero before $T_{\max}$ the run returns a fallback discrepancy, which a smooth kernel weights to effectively zero, and at the configured parameters that is the usual outcome (survival 2.6% of 2000 simulations at the true parameter, 1.7% over 400 uniform prior draws). The observed dataset is itself re-drawn until a trajectory survives, so the target is $p(\theta \mid s_{\text{obs}}, \text{survived})$, which is internally consistent: the extinct particles’ zero weight is the $P(\text{survive} \mid \theta)$ factor of the likelihood. Throughput and scaling are unaffected, because an extinct simulation costs what a surviving one costs. Posterior recovery is affected: the informative sample is $\approx 2\%$ of the nominal budget, so we report Lotka–

Table 13: Calibration sensitivity to the archive size k and the AMIS snapshot buffer S on the d -dimensional Gaussian-mean target (1000 trials per cell; entries are the mean signed deviation of empirical from nominal coverage over the 0.50/0.80/0.90/0.95 levels and the d parameters). Negative is over-confident, positive is conservative; 0.000 is exactly calibrated. **Left:** varying k at $S = 20$. **Right:** varying S at $k = 100$.

archive size k (at $S = 20$)					
	50	100	200	400	800
$d = 1$	-0.014	-0.003	+0.006	+0.006	+0.074
$d = 2$	-0.020	-0.006	+0.009	+0.030	+0.072
$d = 4$	+0.025	+0.064	+0.094	+0.133	+0.063
$d = 8$	-0.250	+0.055	+0.180	+0.195	+0.061
$d = 16$	-0.047	+0.028	+0.049	+0.053	+0.055

snapshot buffer S (at $k = 100$)				
	0	5	20	50
$d = 1$	-0.088	-0.001	-0.003	-0.005
$d = 2$	-0.088	-0.001	-0.006	-0.003
$d = 4$	+0.039	+0.068	+0.064	+0.064
$d = 8$	+0.065	+0.006	+0.055	+0.074
$d = 16$	-0.010	+0.010	+0.028	+0.034

Table 14: The archive size on both of its axes at once. **Throughput:** median asynchronous throughput at the single-node peak (48 workers) on Lotka–Volterra, 180s wall cap, five replicates ($k \in \{100, 1000\}$ from the main scaling run, $k \in \{50, 200, 400, 800\}$ from the matched k -frontier run), and its change relative to the $k=100$ we use throughout. **Calibration:** worst absolute deviation of empirical from nominal coverage over the dimensions of the d -dimensional Gaussian-mean sweep at $S=20$ (Table 13), with the dimension attaining it. **Crossover:** fully packed nodes at which pyABC (population = worker count) overtakes the asynchronous sampler. The sweep did not include $k=1000$.

k	sims/s at $W=48$	vs. $k=100$	worst $ \Delta\text{cov} $	at	pyABC overtakes
50	6684	+7%	0.250	$d=8$	beyond 3 nodes
100	6240	—	0.064	$d=4$	beyond 3 nodes
200	5904	−5%	0.180	$d=8$	beyond 3 nodes
400	5393	−14%	0.195	$d=8$	3 nodes
800	4331	−31%	0.074	$d=1$	3 nodes
1000	3804	−39%	—	—	3 nodes

Volterra as a systems benchmark and treat its posterior curves as diagnostics that bound no method’s attainable accuracy.

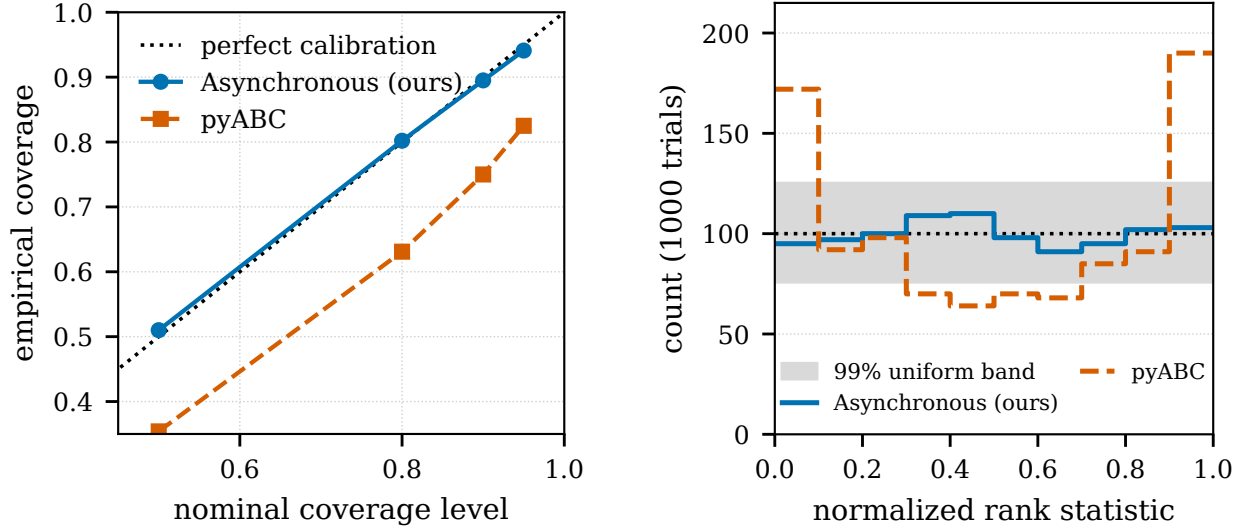


Figure 12: Simulation-based calibration on Gaussian-mean inference (1000 trials; the full-history AMIS posterior): empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.

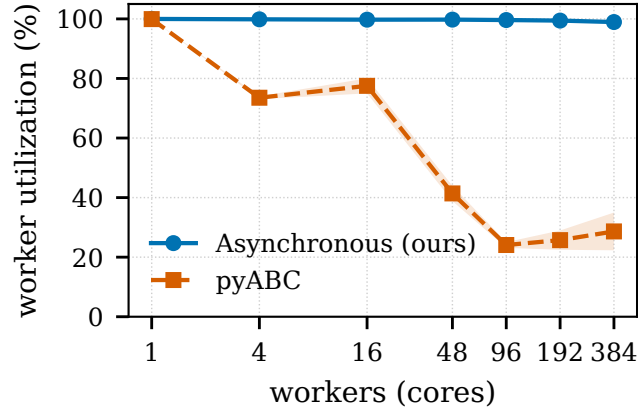


Figure 13: Cellular Potts 50^3 worker utilization, earlier parameterization (fraction of worker wall-clock spent inside the simulator) versus worker count (1800s budget; error bars are ± 1 s.d.); pyABC uses a population equal to the worker count (Fig. 3, right). The asynchronous sampler stays ≈ 98.9 – 99.8% utilized at every scale. pyABC fills every core within a generation but then idles at the barrier waiting for the slowest simulation, so its utilization falls to ≈ 24 – 29% ($41\% \rightarrow 24\% \rightarrow 26\% \rightarrow 29\%$ at 48/96/192/384 workers, i.e. ≈ 70 – 76% idle at every multi-node scale). The figure is the measured utilization behind pyABC’s sub-linear scaling in Fig. 3 (right).